

CS673 Software Engineering

Software Design Part II
Patterns

In Design, We Allocate Responsibilities

- The object-oriented approach to software design is about allocating responsibilities (data, methods) to different classes in the program
 - and to different team members working on different parts of the program
- It is important to keep people's focus of attention as narrow as possible
 - so that they do not keep in mind extra details about other parts of the project

Design Has To Be Communicated

- This means that the developers must be able to describe and discuss their solutions and compare alternatives
 - in a language to be easily understood
- This language should be able to describe each component and its responsibilities
 - and relationships among components
- Design specification is one solution: text
 - it describes data fields, methods, rules of their use, collaborating classes, inheritance

Design Has To Be Communicated

- Textual description is adequate when it describes the properties of one piece
 - however, it is not informative enough to describe relationships between pieces
 - it takes time to write it down in all details and check for consistency in many places
 - it is not pliable enough to be easily changed when different design alternatives are discussed and compared
- Since it specifies all details at the same level, it cannot be used for an overview

UML

Unified Modeling Language

- UML: the Unified Modeling Language
 - diagrammatic representation (model) of relationships among different sets of program entities
- These are several languages used at different stages of development
 - analysis: Case Diagrams (interactions with external entities and their primary steps) and Activity Diagrams (steps of workflow of the program domain)

Unified Modeling Language

- Some other kinds of UML diagrams:
 - design: Class Diagrams (class name, attributes, behavior, relationships) and State Diagrams (possible object states and possible transitions between these states)
 - detailed design: several kinds of Interaction Diagrams (details of how objects interact with each other), e.g., Sequence Diagrams
 - deployment: Deployment Diagrams that show how modules interact when deployed
- It is a huge set of complex languages (hard to learn, v2.5.1 spec is 750 pages)

<https://www.omg.org/spec/UML/>

Unified Modeling Language

- Another way to classify UML diagrams:
 - Structure Diagrams: Class Diagram, Object Diagram, Component Diagram, Composite Diagram, Package Diagram, Deployment Diagram
 - Behavior Diagrams: Use Case Diagram, Activity Diagram, State Machine Diagram
 - Interaction Diagrams: Sequence Diagram, Communication Diagram, Timing Diagram, Interaction Overview Diagram

UML as a Communication Tool

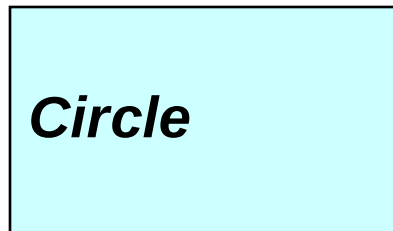
- The UML is first and foremost a communication tool
 - between members of development team
 - between the team and its customers (if customers learn this language) – to solicit better and more complete requirements
 - or even for an individual developer in the process of evaluating design alternatives
- It is easy to overuse the UML
 - and waste a lot of time drawing diagrams

UML as a Communication Tool

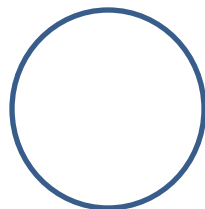
- Used properly, the UML facilitates communications
 - it helps to determine whether all participants have the same understanding of the system and its components
- It allows them to discuss the system at different levels of detail
 - from detailed diagrams showing many implementation-related details to high-level overview of the system in general

Class Diagram

- The most basic UML diagram is the Class Diagram
 - it describes classes that are part of the system (or components of some part of the system)
 - and it shows the relationships among classes and among objects of these classes

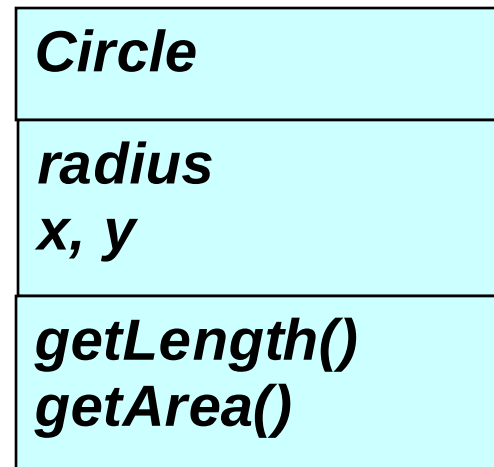
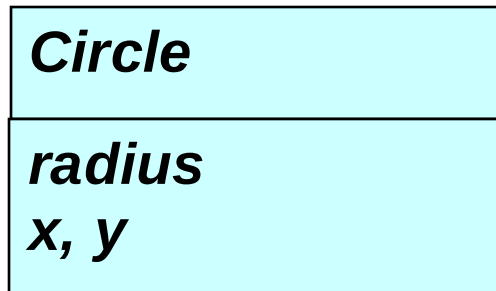


The most basic form of the class diagram contains rectangles with minimum information: just the class name



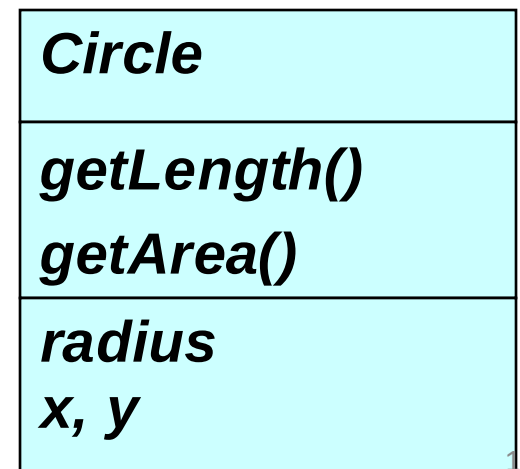
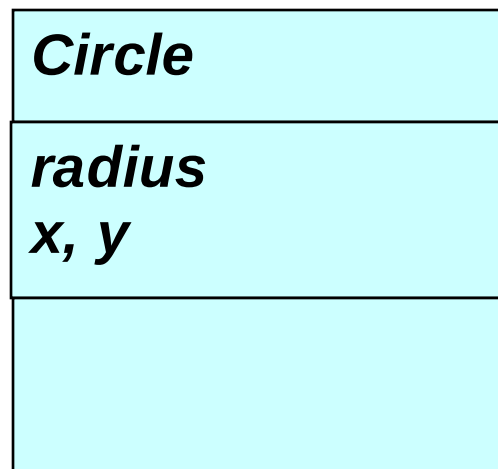
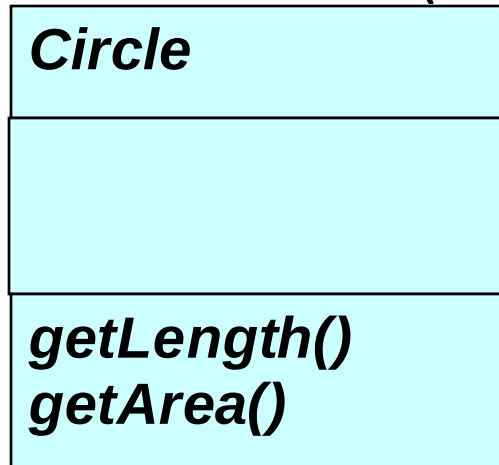
Class Diagram

- A more detailed notation shows design decisions related to
 - data fields of the class
 - the methods of the class
- The rectangle is divided into two or three regions to show data and methods



Class Diagram

- Sometimes the designer is interested in data only or in behavior only
 - it is ok to use a rectangle with three sections but fill only one or two of them
 - the OMT notation swapped the data and behavior sections (GoF use this notation)



Class Diagram

- UML allows the designer to describe the visibility of components (too detailed?)
 - a prefix '-' describes a private component, '+' denotes public, a '#' means protected

Circle

-radius: int

-x, y: int

#circleDisplay(): String

+getLength(): double

+getArea(): double

+toString(): String

+setRadius(r: int): void

In the type notation, the type follows the object; this is the Pascal/Ada convention rather than C/C++ (and reflects the OMG preferences)

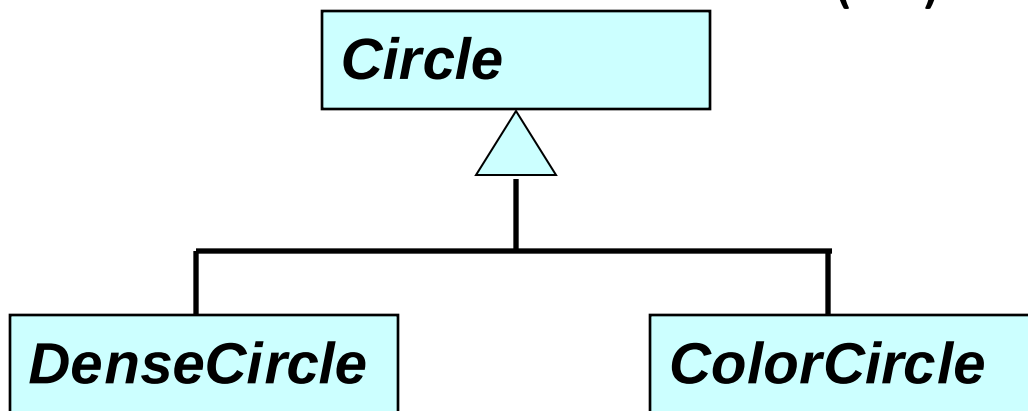
Class Diagram

- The more details we put into the class diagram, the more it approaches the design specification (plain text is better)
 - the less space is left on the page to describe relationships among classes
- When several classes participate in the design under discussion, most designers use just the class name on diagrams describing relationships
 - relationships are represented by drawing lines among squares representing classes

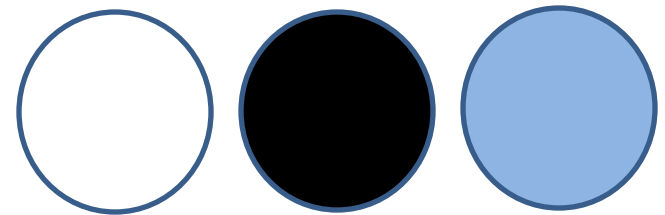
Object Oriented Design

Inheritance Relationship

- The 'is a' inheritance relationship between a superclass and a subclass: an arrow with a hollow arrowhead
 - the arrowhead might be for each subclass or one for all of them; the superclass these days is always drawn above subclass(es)

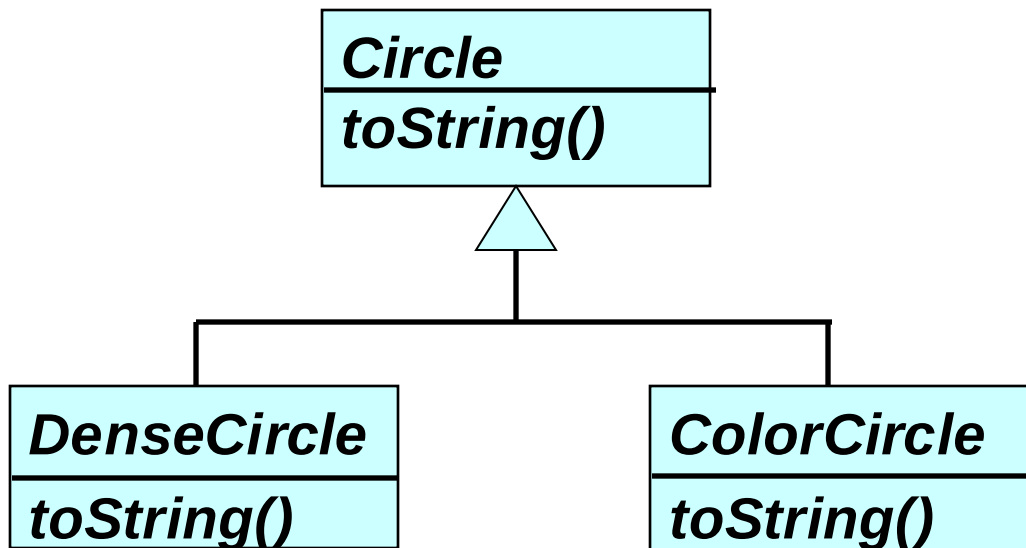


Here, classes DenseCircle and ColorCircle extend class Circle



Inheritance Relationship

- When inheritance is used for specification, the same notation is used
 - the abstract class name is italicized
 - so is the name of abstract methods that are implemented by subclasses

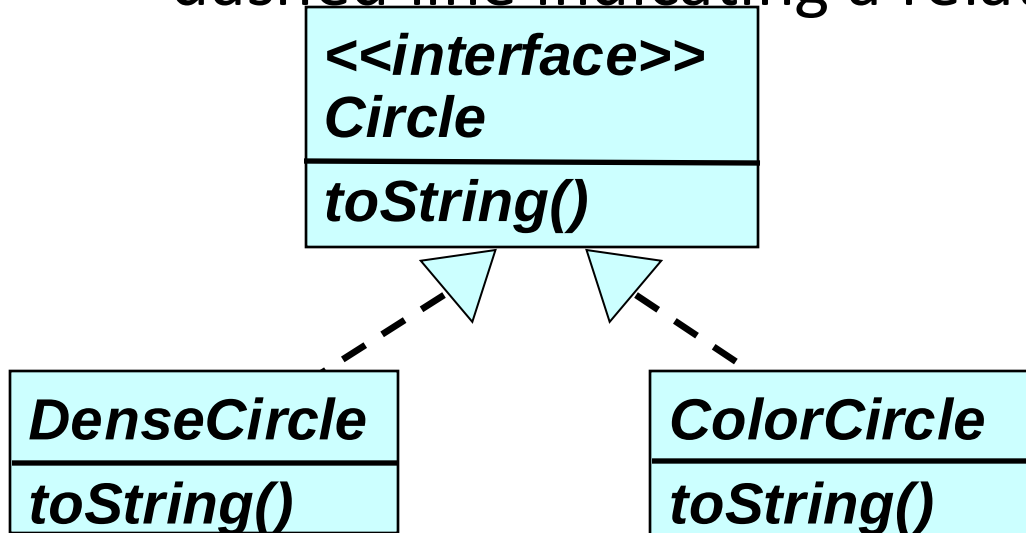


Here, class *Circle* is an abstract class

- the data section is not included
- subclasses have *toString()*, too

Implementing Relationship

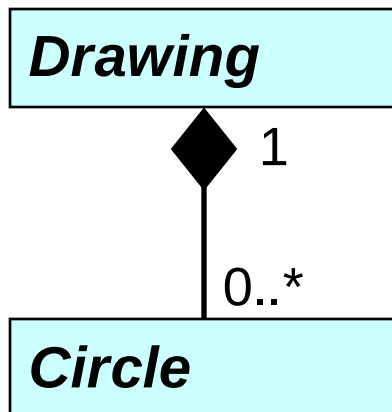
- When the superclass is an interface rather than an abstract class
 - then "subclasses" implement the interface rather than extend the superclass
 - the notation is similar: same arrowhead but a dashed line indicating a relationship



Here, DenseCircle and ColorCircle implement the Circle interface

Composition Relationship

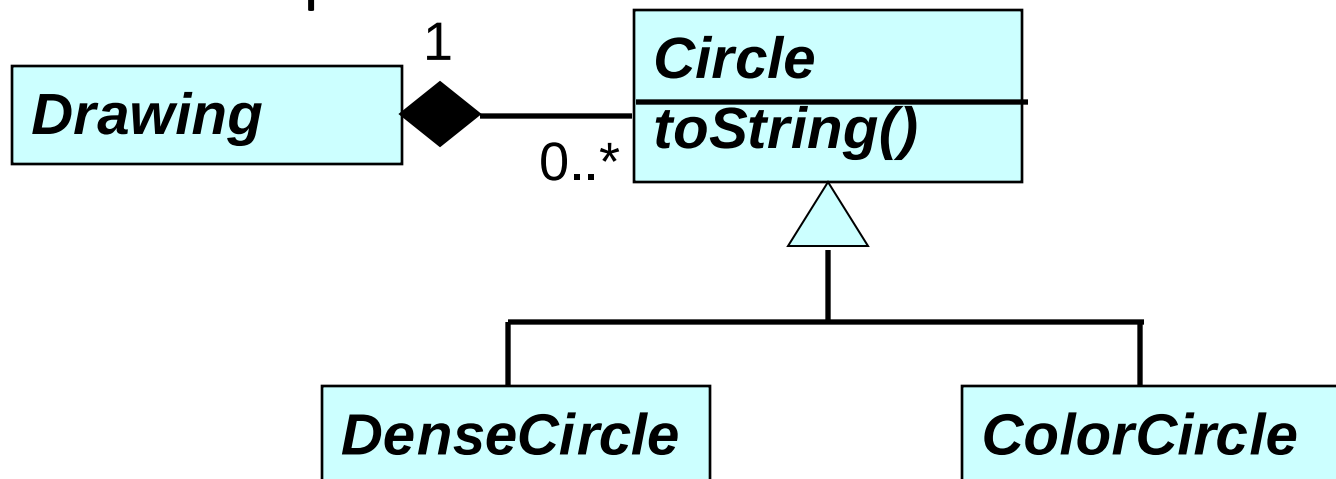
- The 'has a' relationship is used when object of one class has an object of another class as its component
 - the component objects are created when the container (composite) object is created (or some time later); they disappear when the container object disappears



The cardinality notation describes how many objects can participate in the relationship (e.g., 0..n)

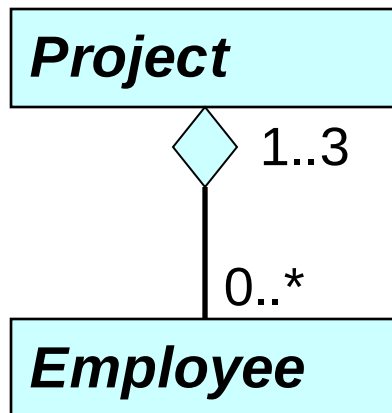
Composition Relationship

- If a component object belongs to the inheritance hierarchy, the whole hierarchy is displayed as a component
 - attention: if the superclass of the hierarchy is an abstract class, only subclass objects can become actual components!



Aggregation Relationship

- The UML calls this relationship between objects (not classes!) composition
 - components exists in the container and nowhere else, not shared with others
- If a component can exist independently of the container, it is called aggregation



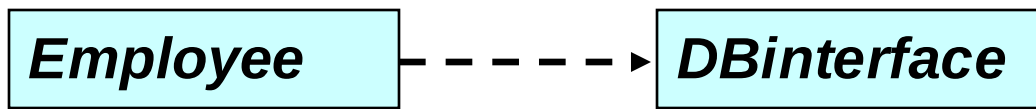
The hollow diamond says that components might be created independently of the container

Aggregation Relationship

- You can view composition as an unshared (exclusive) relationship between the container and components
 - aggregation means that the container can share components with other containers
 - the life cycle of component is not tied to the life cycle of the container (non-exclusive)
- GoF use the reverse terminology (OMT)
 - composition means a shared relationship
 - aggregation means exclusive use
 - their work precedes UML by several years

General Association

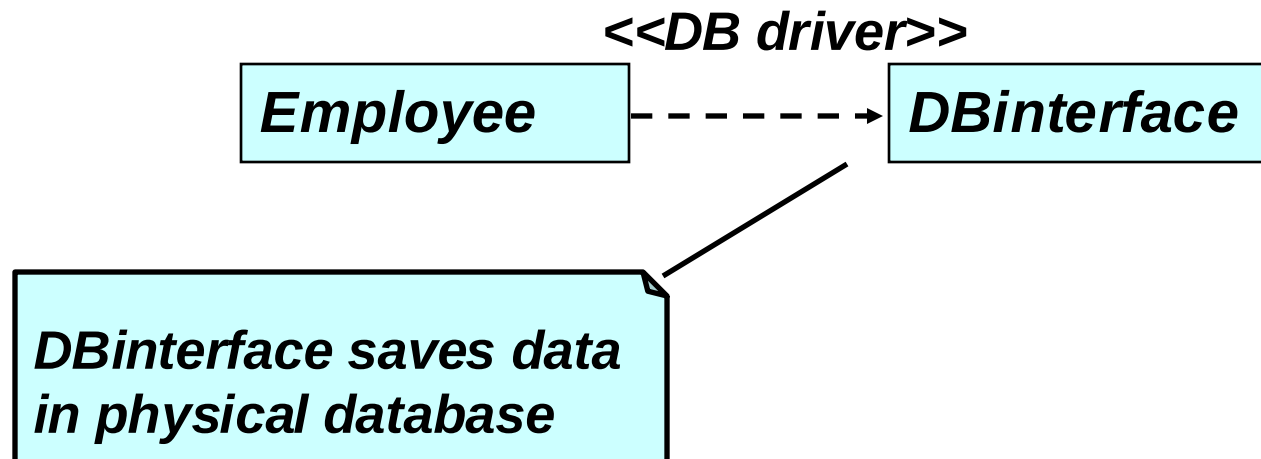
- An unspecified association between classes is denoted by a solid line
- A directional association is denoted by a dashed (or dotted) line with an arrow
 - one object uses another, or depends on another or creates another



Here, DBinterface saves Employee data in the database

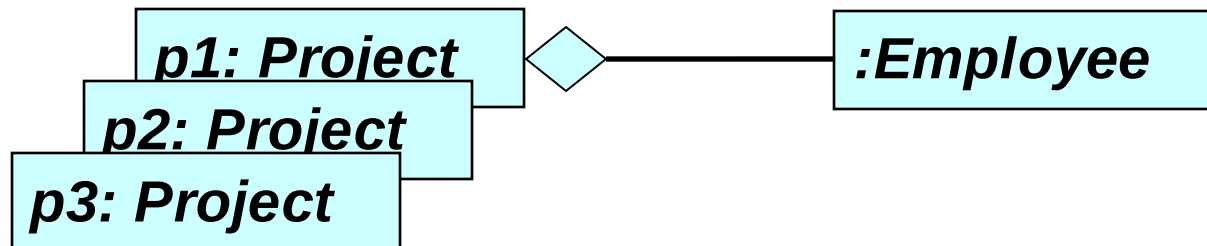
Stereotypes and Notes

- The existence of dependency relationship reflects the existence of a semantic association among objects
 - to specify its meaning we use "stereotypes" (text in double angle brackets) or notes (text in a box with the right corner folded)



Object Diagram

- Class diagrams show static relationships between classes
 - they do not show any activities
- Object diagrams show how many objects of each class exist at a particular moment during execution
 - They include object name and its type or type only (preceded by a semicolon)

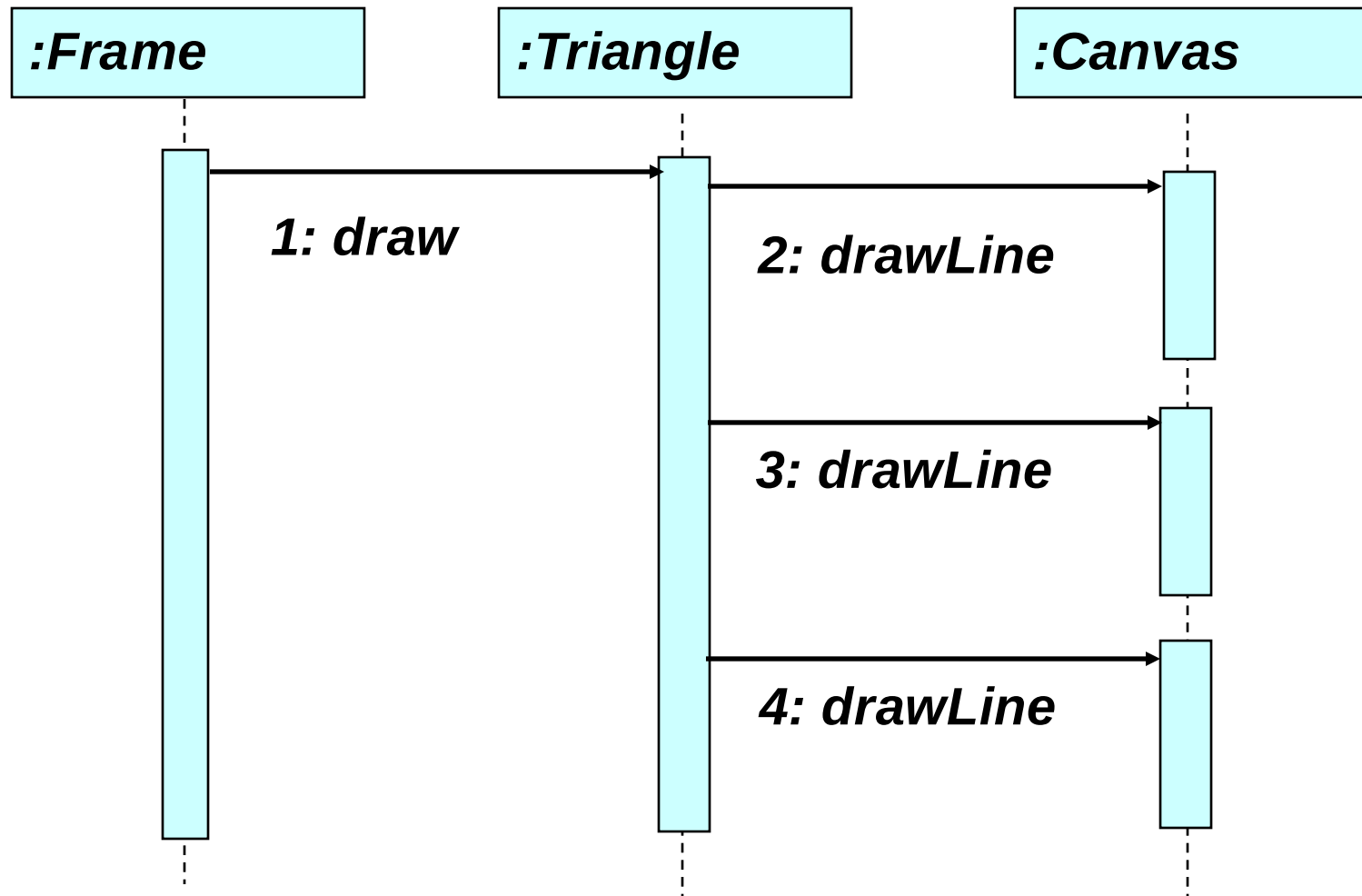


Interaction Diagram

- Another technique for showing actions between objects is using one of the Interaction Diagrams
 - the Sequence Diagram is the most common type of Interaction Diagram
- They are read from top to bottom
 - rectangles on top represent objects
 - same naming convention as for the object diagram: either 'object : type' or just ':type'
 - the vertical lines represent the lifeline of the objects: time goes from top down

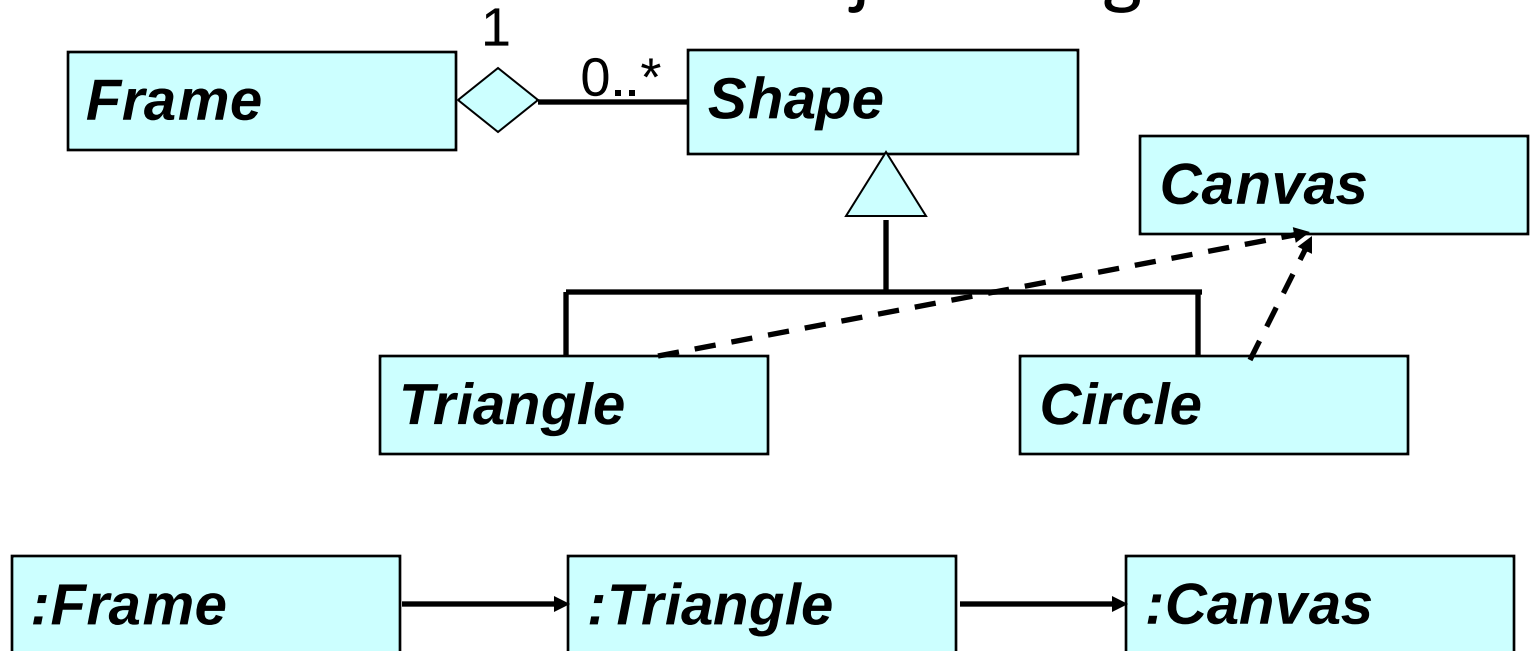
Sequence Diagram

- Horizontal lines represent method calls



Sequence Diagram

- As appropriate for the sequence diagram, it represents the sequence of actions of interest during execution
- Here are a class and an object diagram



Design Patterns

Inheritance as a Design Pattern

- Inheritance is actually a design pattern
 - it is often used alone as a powerful design technique (allocation of responsibilities)
 - it is used in combination with other design patterns (as the major part of design)
- It supports two goals of the object-oriented approach
 - encapsulation of what we hope will be permanent (in the superclass)
 - encapsulation of what we think will change (in subclasses)

Inheritance as a Design Pattern

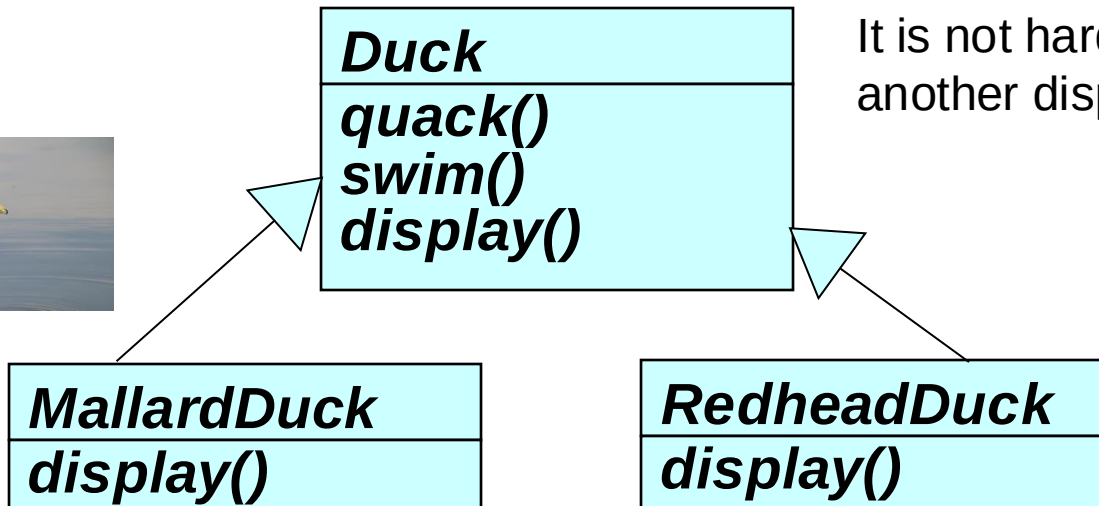
- The variable part usually encompasses different modes of behavior
 - each subclass encapsulates its separate mode of behavior, e.g., `getPay()`
 - they are different for each subclass; still, we use the same method name in all subclasses since they are similar enough
 - when the system has to support yet another mode of behavior, we add another subclass with the same method names
 - this is just beautiful!

Inheritance as a Design Pattern

- When what is expected to stay permanent changes nonetheless
 - it often introduces a new pattern of behavior common to all subclasses (e.g., method `changeName()`)
 - we add it to the superclass and this behavior is automatically propagated to all subclasses (inheritance for extension)
 - this is also beautiful!
 - but we have a problem when this behavior cannot be applied to all subclasses

An Example: a Simple Inheritance

- Example: a duck pond simulation game
 - common behavior includes `quack()`, `swim()` and `display()`
 - however, different kinds of ducks display differently on the screen;
 - still, these differences are small enough

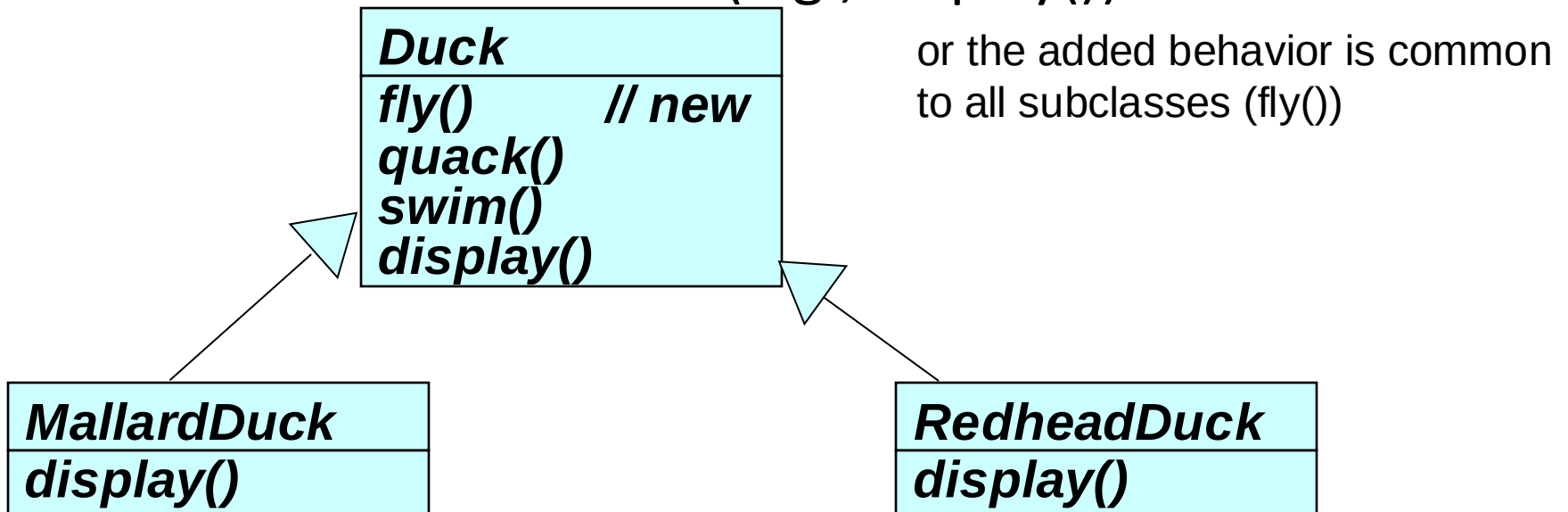


It is not hard to add a duck with another `display()` behavior



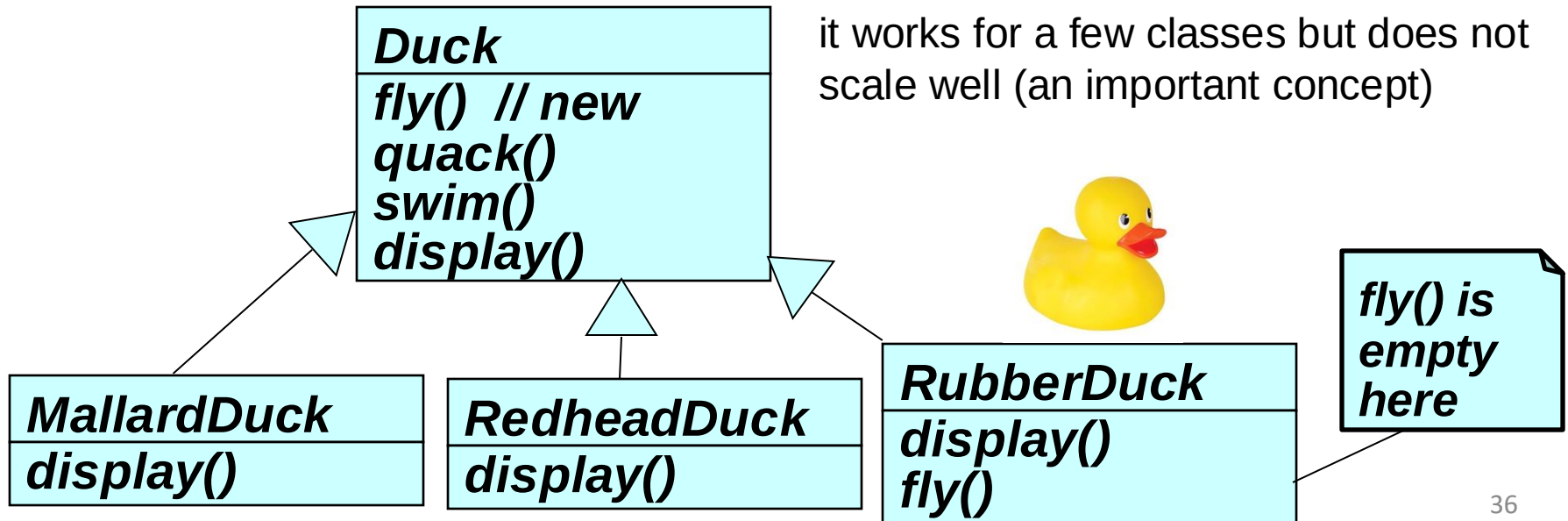
An Example: a Simple Inheritance

- If we need to add behavior common to all kinds of objects, we add it to superclass
 - indeed, inheritance is a great tool for reuse
 - as long as behavior varies little so that the same name can be used (e.g., `display()`)



An Example: a Simple Inheritance

- If a new Duck subclass adds a new dimension of variance (e.g., it cannot fly)
 - the use of inheritance becomes awkward
- We can override fly() in the subclass as an empty method: this is ok but very ugly

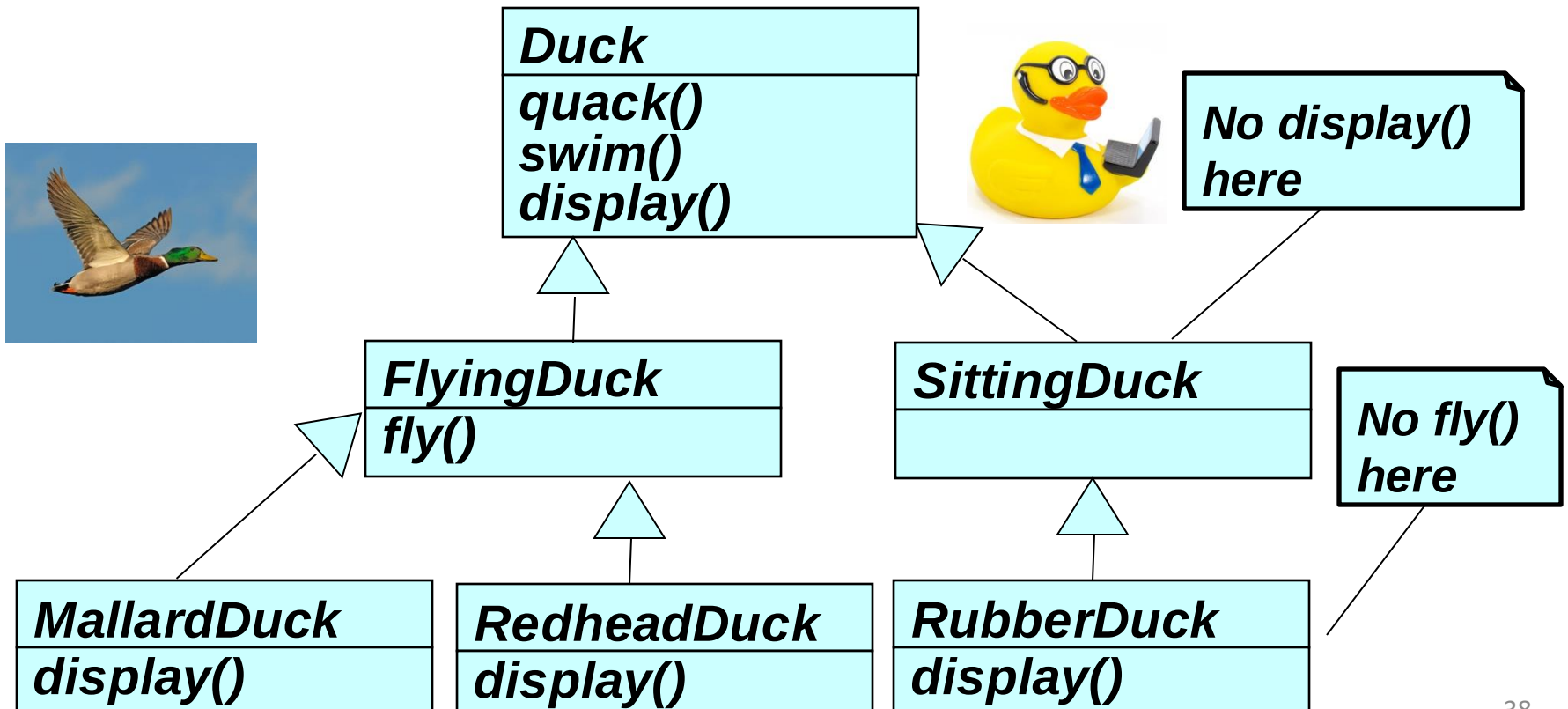


Limitations of Inheritance

- If behavior varies among all subtypes but remains essentially the similar
 - inheritance is a sufficient design pattern
- If behavior applies to some subclasses but not to all, inheritance leads to using a large number of empty methods
 - the number of aspects of variance and exceptions can become large (some ducks do not fly, others do not quack, and so on)
- As a community, we use inheritance too much; this results in bad designs

Limitations of Inheritance

- Another way: make the hierarchy taller
 - encapsulate the new aspect in additional levels of the hierarchy

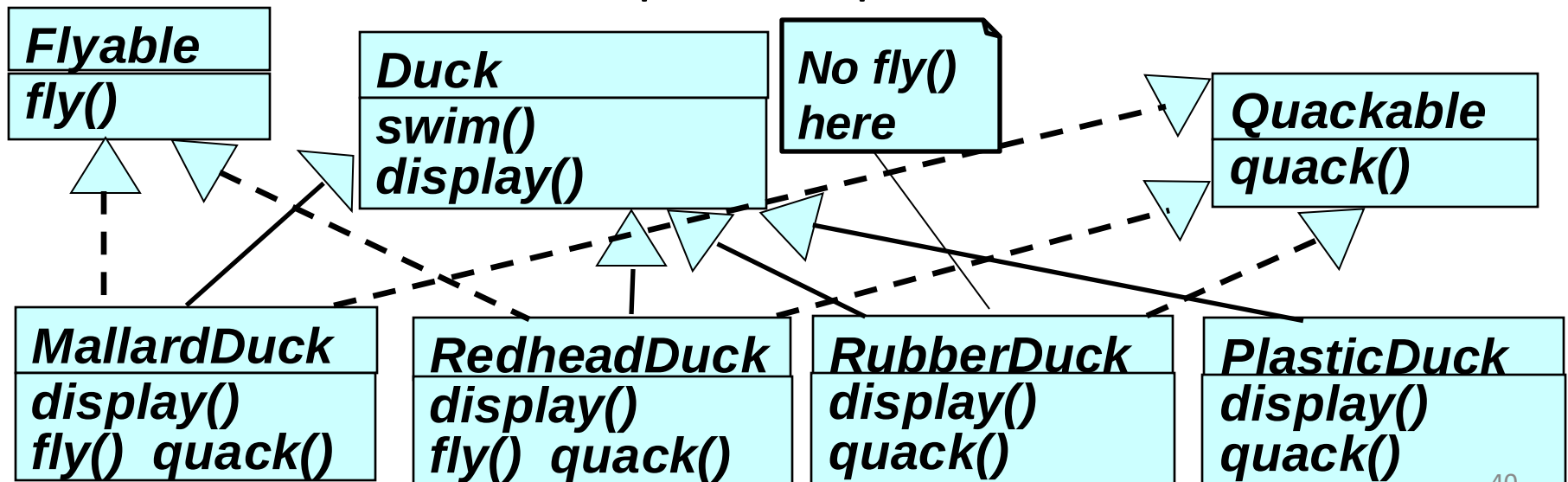


Limitations of Inheritance

- So, adding new class RubberDuck requires changes in all classes in the inheritance hierarchy and adding more classes
 - this is doable, but scalability is poor
- If we need to vary yet another aspect, e.g., the way the ducks quack (some do not, some squeak) we override this method in each subclass
 - if the behavior deserves another name (e.g., squeak instead of quack) we add yet another intermediate level of inheritance
 - and what if a class has to belong to two sub-hierarchies (it squeaks but flies)?

Limitations of Inheritance

- One possible approach to the growth of empty overrides is the use of interfaces
 - since not all ducks quack or fly, we can define interfaces, e.g., Flyable, Quackable
 - those ducks that fly implement Flyable
 - the ducks that quack implement Quackable



Limitations of Inheritance

- This is a nice idea, but it leads to bad design: duplicate code and multiple changes during maintenance
 - ducks that fly in the same way duplicate fly behavior (both data and method code)
 - changes to this behavior require changes in each subclass that implements fly()
- Interface is a mechanism of coercion
 - its use forces the subclass to implement a method, but it results in duplication of data and code across subclasses (poor reuse)

Encapsulate All That Vary

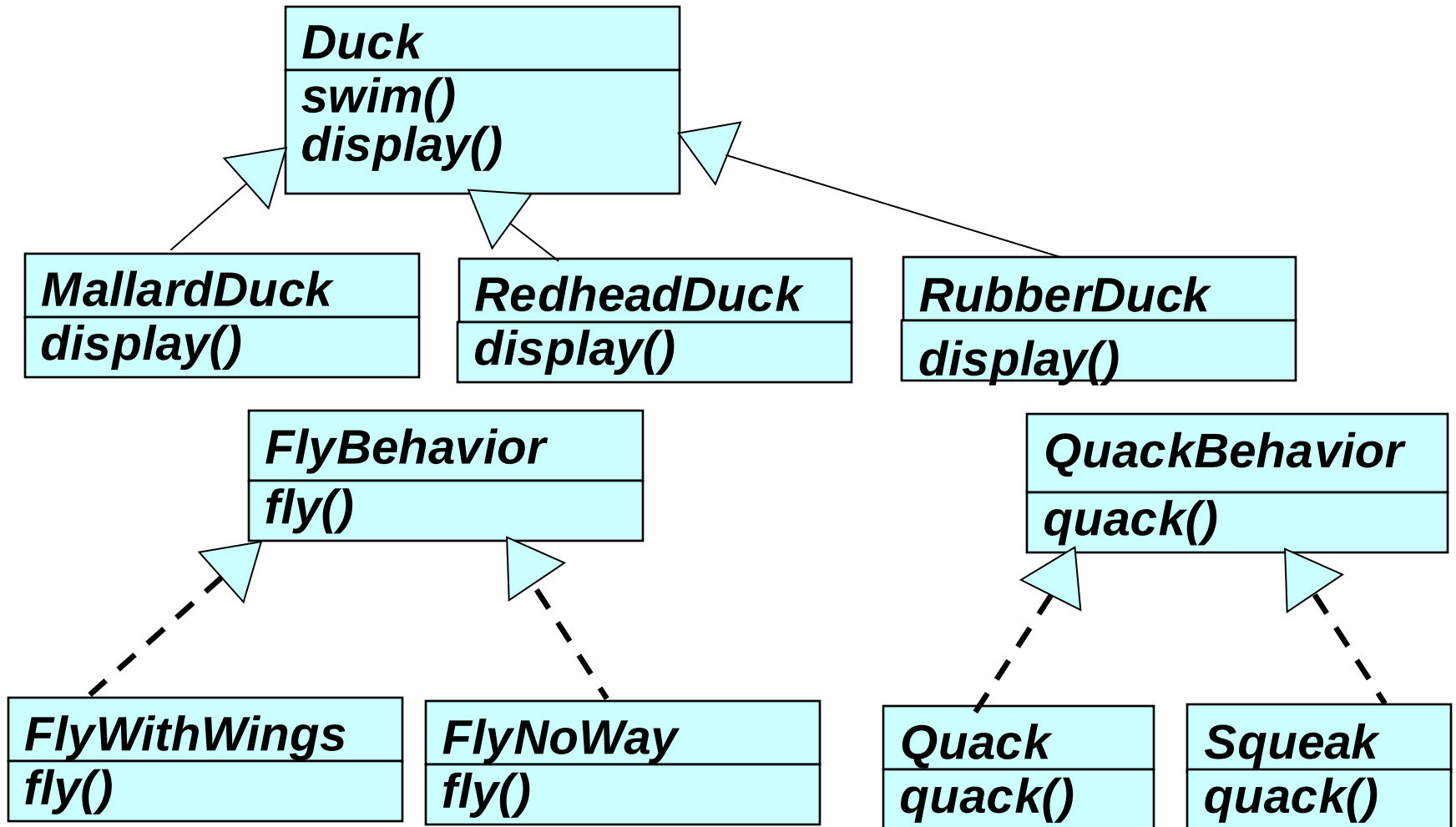
- Design principle: identify the aspects of the application that vary and separate them from what stays the same
 - encapsulate it so that it could change without affecting other parts of design
 - another form of a) info hiding, b) separation of concerns, c) using separate abstraction levels, d) pushing work down to servers, or e) low coupling: a more flexible system
- Ultimately, everything can change
 - but some things are more stable and others change relatively soon (subjective)

Encapsulate All That Vary

- Inheritance is still a great design technique: it encapsulates variability
 - but each inheritance hierarchy should encapsulate one aspect of variability only
- Our trouble stems from an attempt to add two aspects to the same hierarchy
 - flying behavior and quacking behavior
- Instead, let us create a hierarchy for each aspect of variability
 - flying behavior with its subclasses
 - quacking behavior with its subclasses

One Varying Aspect per Hierarchy

- Here, each aspect has its own hierarchy



One Varying Aspect per Hierarchy

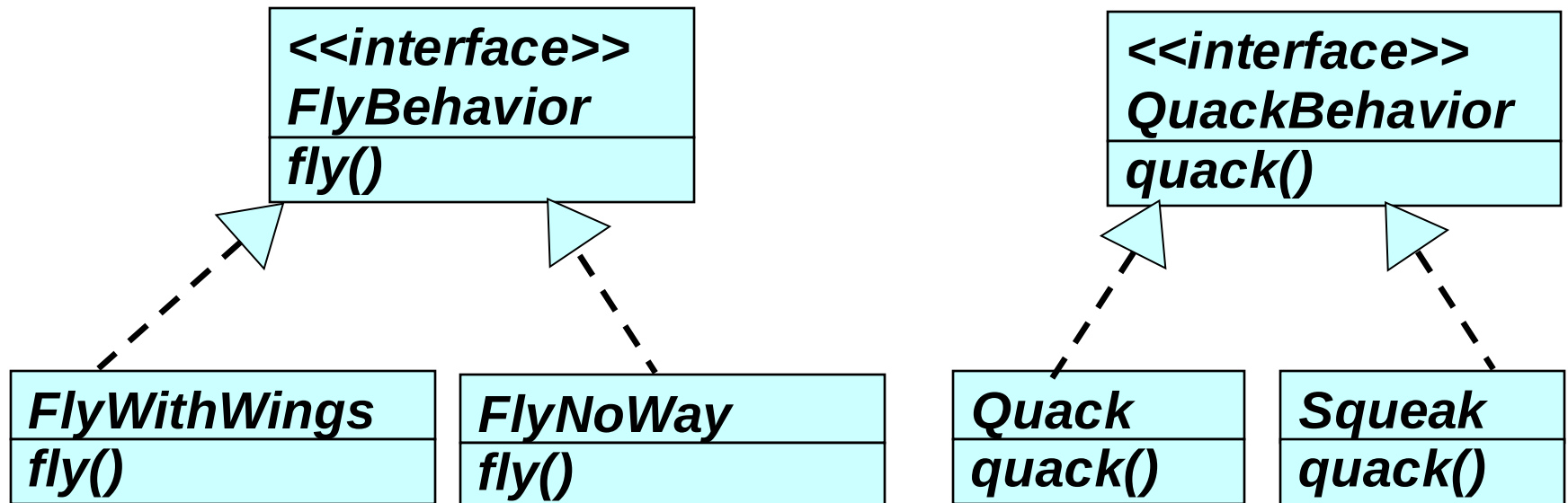
- Now methods `fly()` and `quack()` should be eliminated from the Duck superclass
 - not all Duck subclasses should inherit these methods: some will override them in their own way, some do not have them
- Instead, we create two other hierarchies
 - the flying behavior hierarchy will have subclasses with method `fly()` for flying with winds, flying like a rocket, or not flying at all
 - the quacking behavior hierarchy will have subclasses for quacking, squeaking, etc.

One Varying Aspect per Hierarchy

- One of side benefits will be the ability to change object behavior at run time
 - actually, we did not ask for this flexibility
 - this will impose on the client (simulation program) an additional job to specify behavior even when it is stable and can be specified in server classes (duck objects) themselves
- We use here the important design principle: program to an interface, not to implementation, use superclass pointers

One Varying Aspect per Hierarchy

- In this new design, superclasses only specify behavior (as interfaces or as abstract classes)
 - actual specific behavior is implemented by subclasses that implement the interfaces

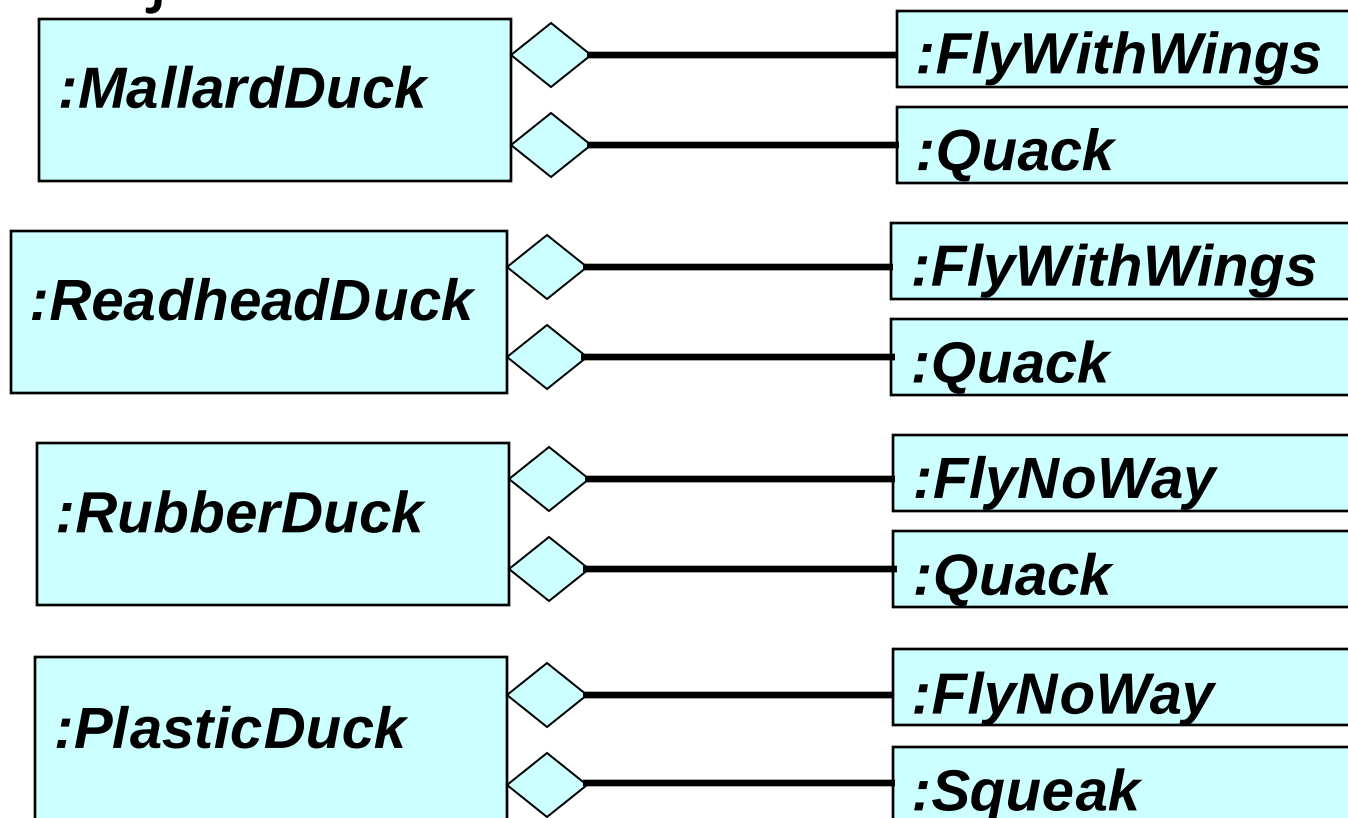


Link Hierarchies with Aggregation

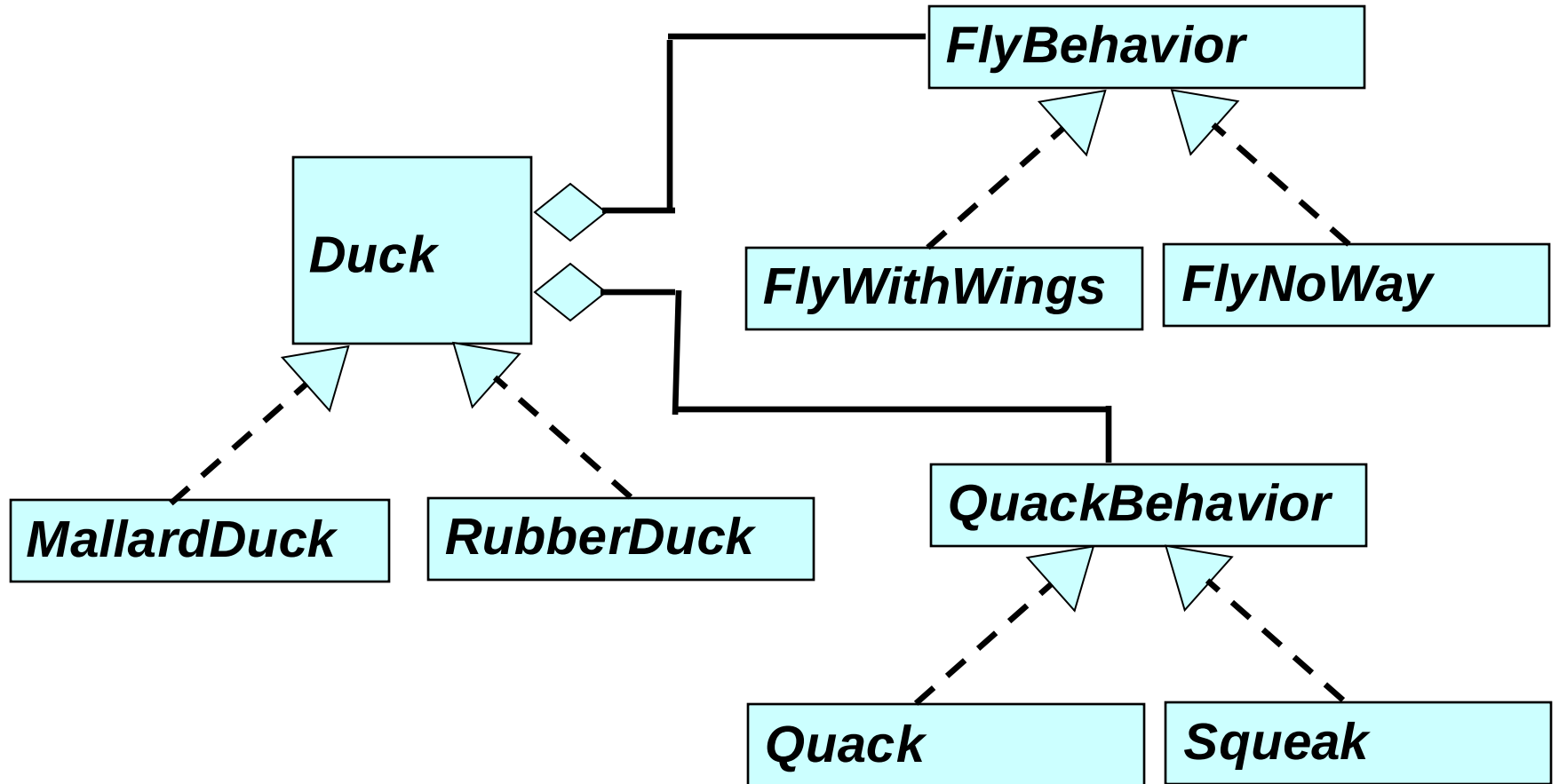
- Now we have three inheritance hierarchies for three aspects of change
 - for a) display, b) flying, and c) quacking
 - how do we link them together to specify which duck flies and which does not, which duck quacks and which one squeaks?
- The answer is: through aggregation
 - each Duck object has its own FlyBehavior object: either FlyWithWings or FlyNoWay
 - also it has its own QuackBehavior object: either of type Quack or of type Squeak

Link Hierarchies with Aggregation

- Here is an object diagram for four concrete objects



Link Hierarchies with Aggregation



Link Hierarchies with Aggregation

- We separated the Duck classes from variations in `fly()` and `quack()` behaviors
 - one advantage: these behavior objects can be reused as components of other classes
 - another advantage: new behaviors could be added to the Duck hierarchy of classes without modifying existing behavior classes
- Principle of programming to an interface
 - interface references can point to any concrete object: Duck subclasses do not know what objects they use, concrete objects implement superclass interfaces

Code to Interface vs Implementation

- Programming to 'interfaces' here means specifying the required set of methods and their signatures: e.g., fly(), quack()
 - it can be done either with a Java interface or with an abstract class: same results
 - we use type substitution: we define fields as 'supertype' references, but they point to 'subtype' objects (concrete implementation)
 - the class that defines the references does not know what type of object will be used
 - here, it is known to subclass constructor

How Easy Is It to Extend?

- If we need to add another aspect that varies among the Duck subclasses
 - We do not expand the Duck hierarchy
- Instead, we add another hierarchy of behavior classes, e.g., a PeckBehavior interface and classes that implement it
 - Then we add another component (interface reference) to the Duck superclass
- Total Duck object behavior is composed from its concrete component objects

Two Diagnostics For Good Design

- Single Responsibility Principle (**SRP**)
 - Each class should be responsible for one thing (capability, entity, computation, etc.)
 - Can phrase this as “mind your own business”
 - Object do its own calculations
 - Object should not do calculations for another
 - Easy to violate this because objects need to be connected to one another
 - e.g., Events happen as part of Dates
- Don't Repeat Yourself (**DRY**)
 - Each computational idea should be expressed just once
 - Violations often the result of
 - Cut-and-paste programming
 - Incomplete class (others have to do calculations for it, which also violates SRP)
 - But also over-specialization of classes (class = object)
- Separate Variants from Invariants

Where we stopped

Design Patterns

- High level ideas on how to structure classes and objects to solve certain problem. We need write our own code to adopt those patterns.
- Provide a shared vocabulary
- Framework and libraries are not design patterns but are specific implementations that can be used in our code
 - However, they may make use of design patterns in their implementations

The Motivation for Patterns

- We use the concepts of encapsulation, abstraction, inheritance, polymorphism
 - they alone do not lead to design that is flexible, reusable, scaleable, maintainable
 - we should identify what might change and encapsulate it from the rest of the system
 - limit the use inheritance, use composition
 - code to interfaces, not to implementation
- These are the concepts that are often used in object-oriented design

The Motivation for Patterns

- To facilitate the use of these techniques software industry developed a concept of design patterns
 - they are based on best practices of experienced designers in many application areas
 - they are not coding idioms or programming guidelines or reusable coding patterns
- We define relationships between classes regardless of their internal detail

The Origin of Design Patterns

- We use Design Pattern names rather than describe class relationships
 - this helps designers to communicate their ideas effectively
- The GoF book was the first to catalog design patterns observed most often
 - they claim that they borrowed their approach to design from architecture
 - in architecture, patterns describe typical problems and solutions to these problems that are repeated in many projects

The Origin of Design Patterns

"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice."

*Christopher Alexander, "The Timeless Way of Building"
as quoted by Gamma et al.*

The Origin of Design Patterns

- Experienced software designers use general principles in building programs
 - but they also try to reuse solutions that proved to be successful
 - "successful" means that the design is flexible, understandable, modifiable, expandable, scalable, and maintainable
 - that is, consists of relatively independent cohesive pieces encapsulating specific levels of abstraction
 - some add to this the concept of "elegance"

The GoF Pattern Catalogue

- In their catalog, the GoF classified patterns in three broad categories
 - creational, structural, behavioral
 - the Strategy Pattern is a behavioral pattern
- Another way to classify patterns is whether they describe relationships among classes or among objects
- Creational patterns describe the process of object creation
 - they defer some work of object creation to subclasses or to other objects

The GoF Pattern Catalogue

- Structural patterns describe structural (compile time rather than runtime) relationships of classes or objects
 - as class patterns, they use inheritance to define classes; as object patterns, they describe ways to assemble objects
- Behavioral patterns describe ways in which classes or objects interact
 - for classes, they rely on inheritance to support the algorithms, for objects, they rely on general association or aggregation

The GoF Pattern Catalogue

- The GoF used a combination of four elements to describe design patterns
 - for each pattern, they specified the name, the problem, its solution and consequences
- The pattern name: a cryptic analogy
- The problem: when to apply the pattern
- The solution: the elements of design
 - their arrangement in most abstract but broadly applicable form
- The consequences: results, tradeoffs

The GoF Pattern Catalogue

- As the catalogue framework, they used a super-pattern to describe patterns
 - pattern name and class to which it belongs
 - intent: what design issues does it address?
 - also known as: names used by others
 - motivation: an example that illustrates the problem and how the pattern solves it
 - applicability: situations where it is helpful
 - structure: class and interaction diagrams
 - participants: classes and objects used
 - collaborations: how participants do the work

The GoF Pattern Catalogue

- Five more bullets to go
 - consequences: how the pattern achieves its goal, the trade-offs, costs and benefits, what aspects can be varied independently
 - implementation: pitfalls, language issues
 - sample code: fragments to illustrate ideas
 - known uses: domains where the pattern was used successfully
 - related patterns: other patterns related to this one, differences and similarities

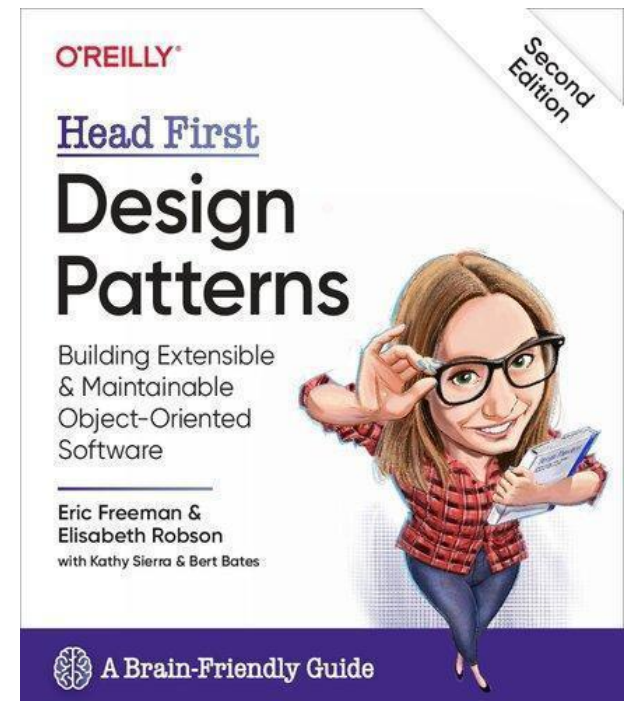
References



From left to right: Ralph Johnson, Richard Helm, Erich Gamma, and John Vlissides. (Gang of Four)

http://en.wikipedia.org/wiki/Design_Patterns

Fun review
and some
good
examples



Eric Freeman,
Elisabeth Robson

An Example Design Pattern

MVC Architecture

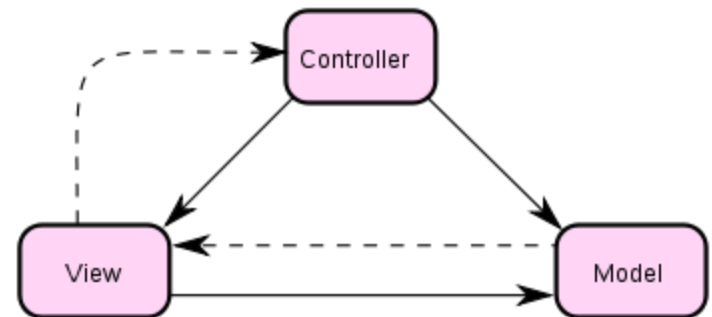
- Separate the business logic and application data from the presentation data to the user.
- Model: business logic, application data
- View: presentation of model, UI
- Controller: mediate between model and view
 - Intercept the request from view and pass to the model for proper actions.
 - Notify the views of the changes in the model.
- Model and view are independent to each other.
- In GUIs, the views and the controllers often work very closely together.
- Reusable and expressive.

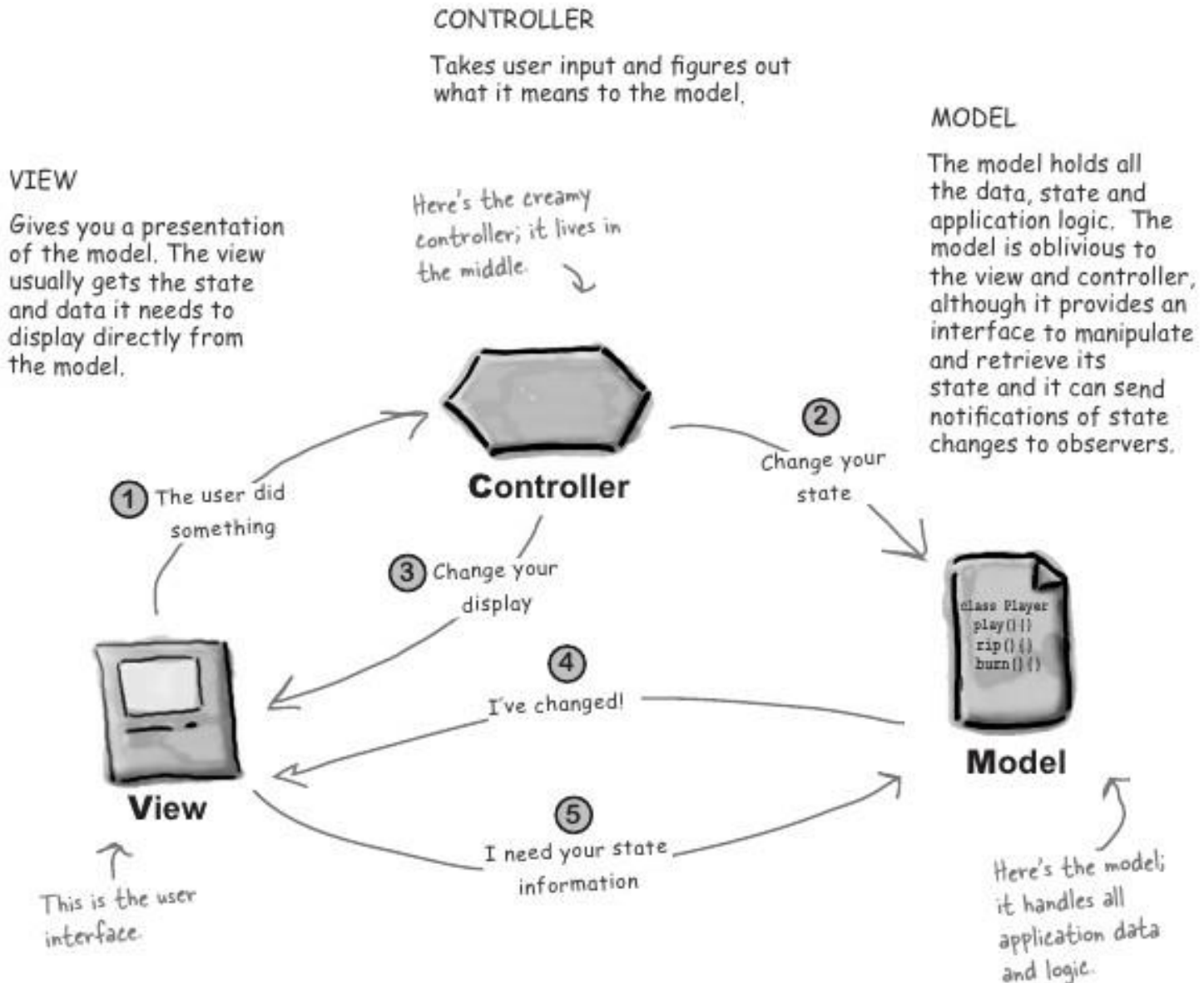
MVC Architecture

- Separate the business logic and application data from the presentation data to the user.
- Model: business logic, application data (entity objects)
- View: presentation of model, UI (boundary objects)
- Controller: (control objects)
 - Intercept the request from view (UI) and pass to the model for proper actions.
 - Notify the views of the changes in the model.
- Model and view are independent to each other.
- In GUIs, the views and the controllers often work very closely together.
- Reusable and expressive.

MVC

- First implemented by Trygve Reenskaug at 1979 on Smalltalk at Xerox labs.
- Used widely
 - GUI, web applications
- Compound Design pattern
- Used in a lot of frameworks
 - Java swing, ruby on rails

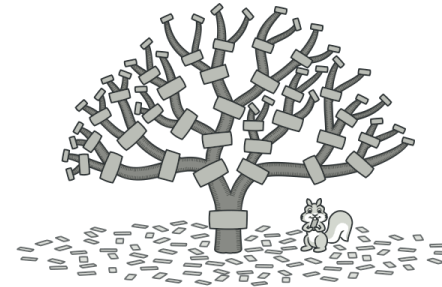
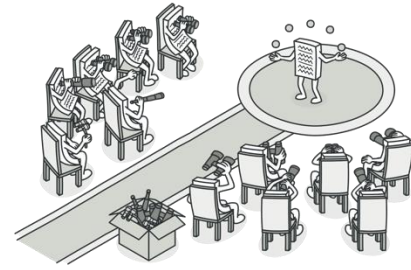




From "Head First Design Patterns"

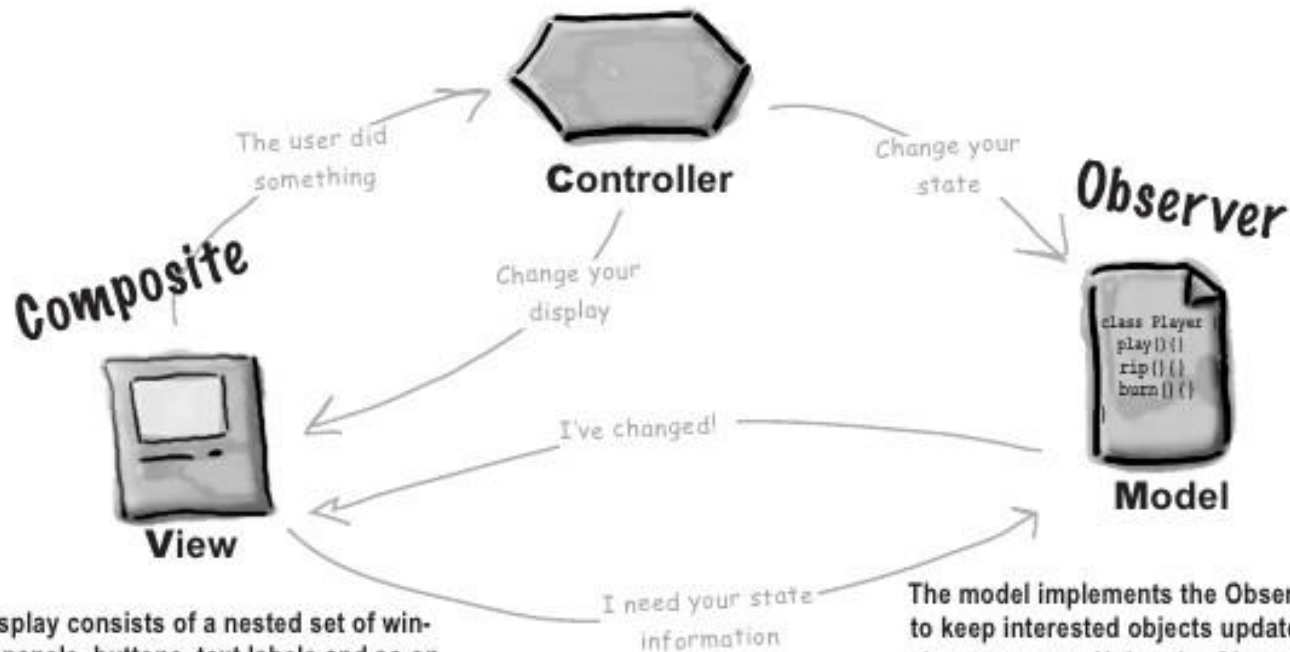
MVC Patterns

- Compound design pattern
 - Observer Pattern
 - Strategy Pattern
 - Composite Pattern



Strategy

The view and controller implement the classic Strategy Pattern: the view is an object that is configured with a strategy. The controller provides the strategy. The view is concerned only with the visual aspects of the application, and delegates to the controller for any decisions about the interface behavior. Using the Strategy Pattern also keeps the view decoupled from the model because it is the controller that is responsible for interacting with the model to carry out user requests. The view knows nothing about how this gets done.

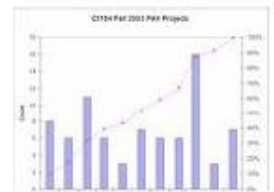
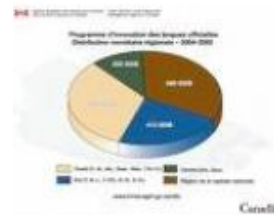
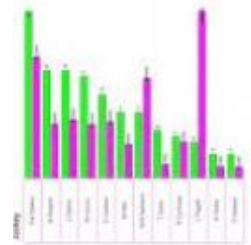
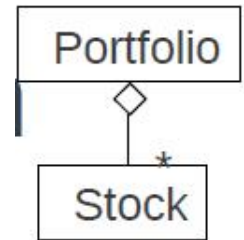


The display consists of a nested set of windows, panels, buttons, text labels and so on. Each display component is a composite (like a window) or a leaf (like a button). When the controller tells the view to update, it only has to tell the top view component, and Composite takes care of the rest.

The model implements the Observer Pattern to keep interested objects updated when state changes occur. Using the Observer Pattern keeps the model completely independent of the views and controllers. It allows us to use different views with the same model, or even use multiple views at once.

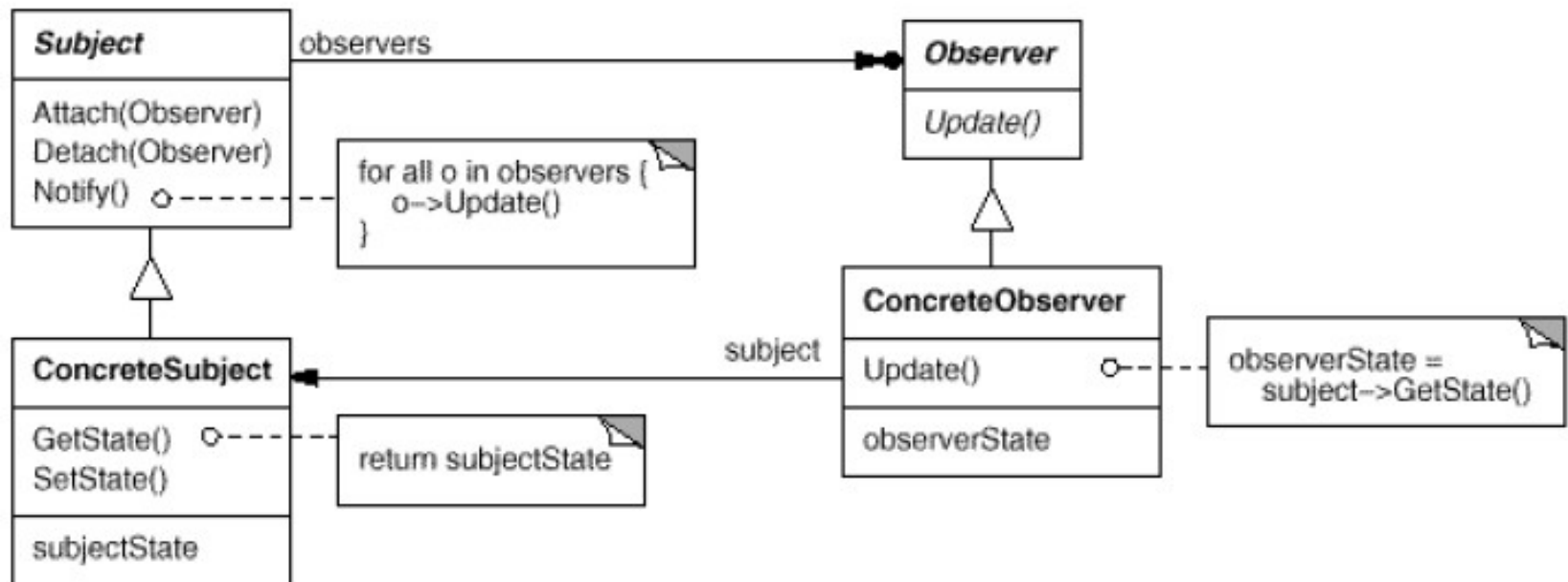
Observer Pattern Motivation

- We have an object that changes its state quite often
 - Example: A Portfolio of stocks
- We want to provide multiple views of the current state of the portfolio
 - Example: Histogram view, pie chart view, time line view, alarm
- Requirements:
 - The system should maintain consistency across the (redundant) views, whenever the state of the observed object changes
 - The system design should be highly extensible
 - It should be possible to add new views without having to recompile the observed object or the existing views.



Observer Pattern

- When you need to write a dependency between objects so that one object can notify its (many) dependents of a state change
- Weather station; Stock report update



3 Possibilities to change the file name

Example

The screenshot shows a Mac OS X desktop environment. On the left is the 'L9_DesignPatterns2.ppt Info' window. The main window is titled 'Lectures' and shows a list of files in 'List View'. A red box highlights the file 'L9_DesignPatterns2.ppt' in the list. Below the list, another red box highlights the file 'L9_DesignPatterns2.ppt' in the 'Open' dialog. On the right, a PowerPoint window is open, displaying a slide titled 'Example: The File Name of a Presentation' and '3 Possibilities to change the File Name'. A blue box on the slide contains the text: 'What happens if I change the file name of this presentation in List View to foo?'. Three blue callout bubbles are present: one labeled 'List View' pointing to the file list, one labeled 'PowerPoint View' pointing to the PowerPoint window, and one labeled 'Info View' pointing to the file info window.

List View

PowerPoint View

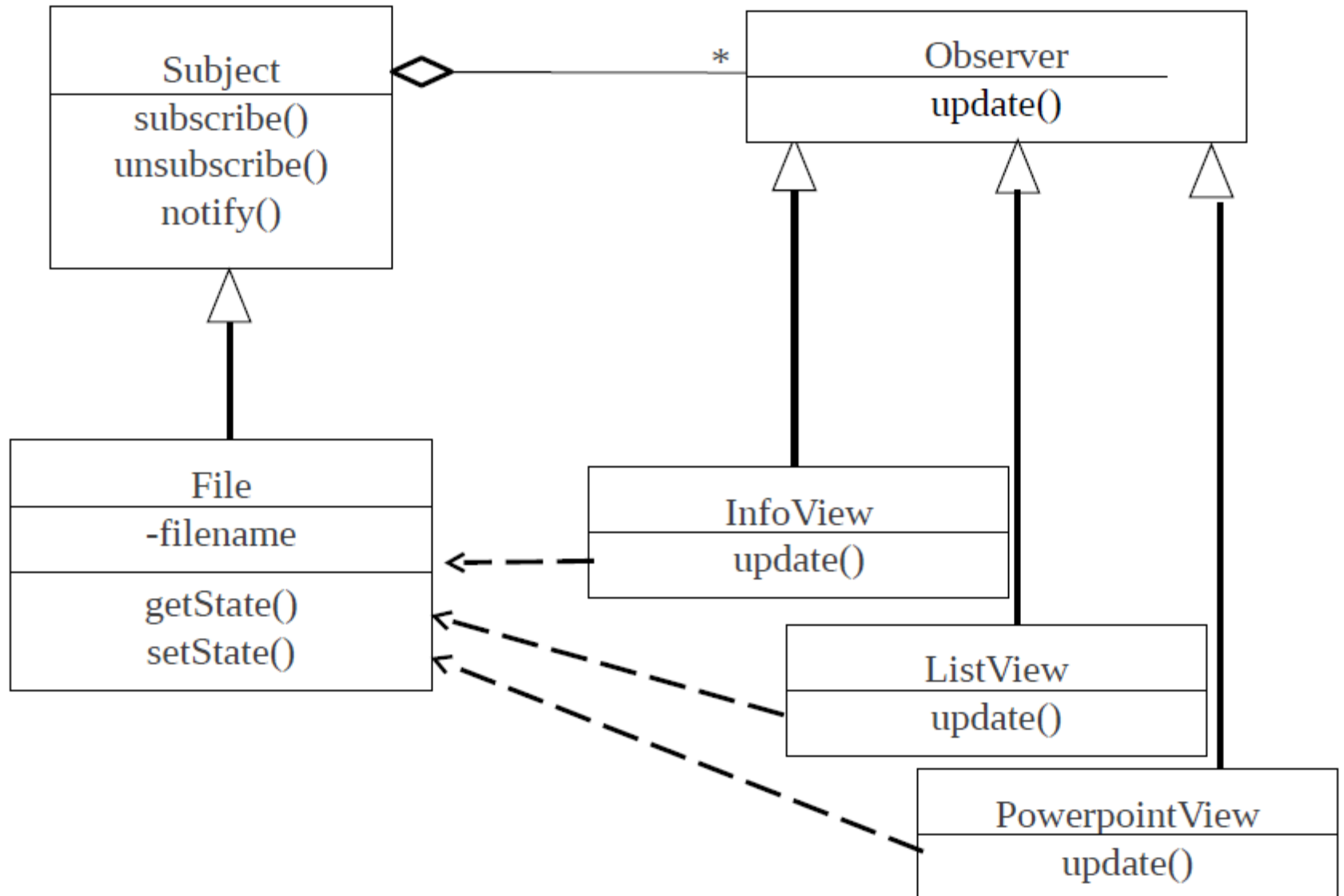
Info View

Example: The File Name of a Presentation
3 Possibilities to change the File Name

What happens if I change the file name of this presentation in List View to foo?

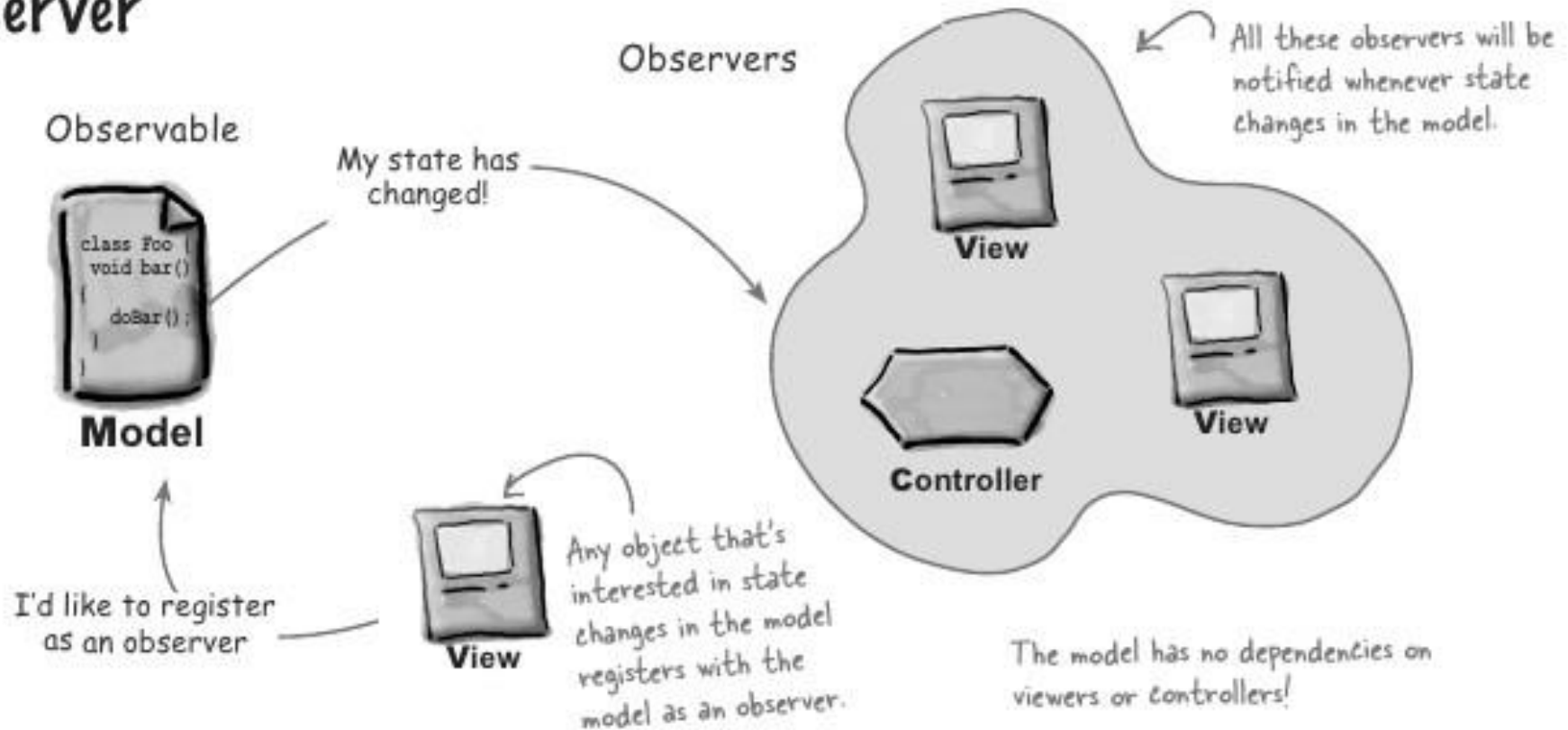
© 2007 Bernd Brügge
Introduction into Software Engineering Summer 2007
Slide 59 of 90

Example



Observer

Observer

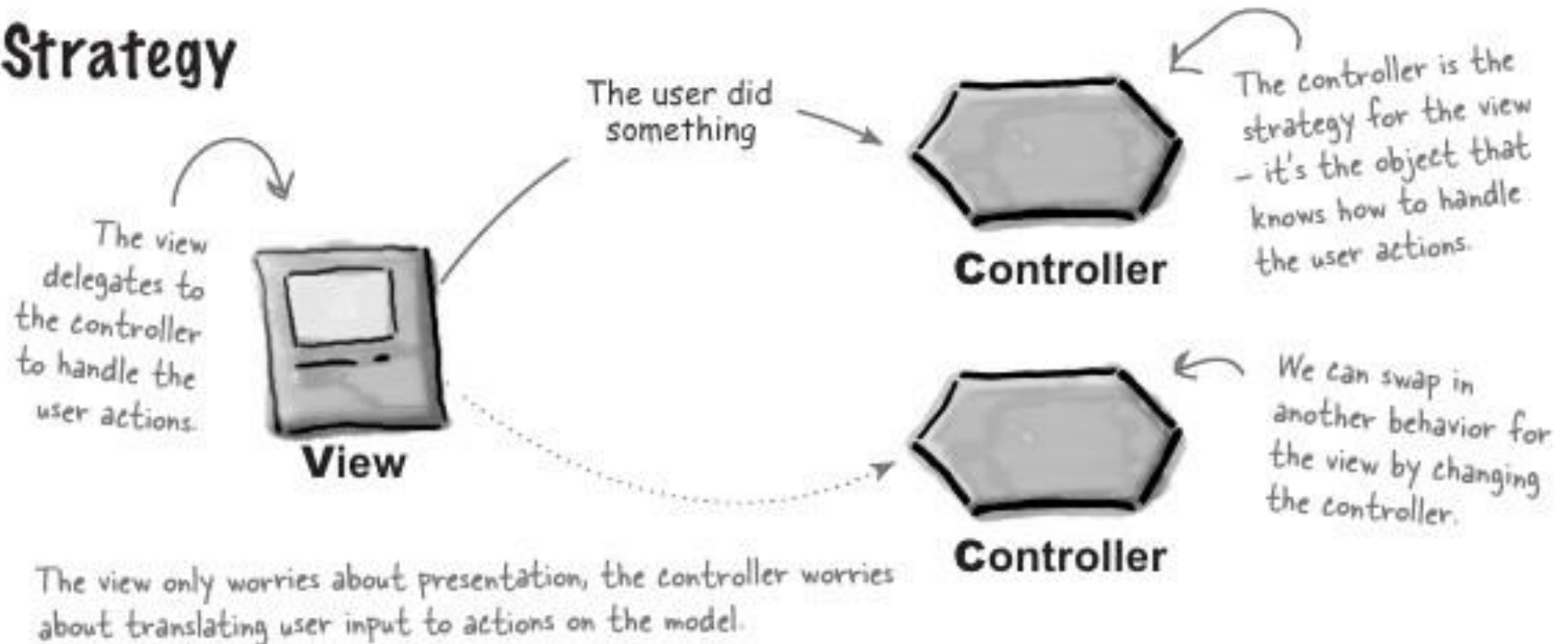


Strategy Pattern Motivation

- Different algorithms exist for a specific task, we can switch between the algorithms at run time.
- Examples of tasks:
 - Different collision strategies for objects in video games
 - Parsing a set of tokens into an abstract syntax tree (Bottom up, top down)
 - Sorting a list of customers (Bubble sort, mergesort, quicksort)
- Different algorithms will be appropriate at different times
- If we need a new algorithm, we can add it without disturbing the application or the other algorithms.
- Strategy provides a way to configure a class with one of many behaviors.

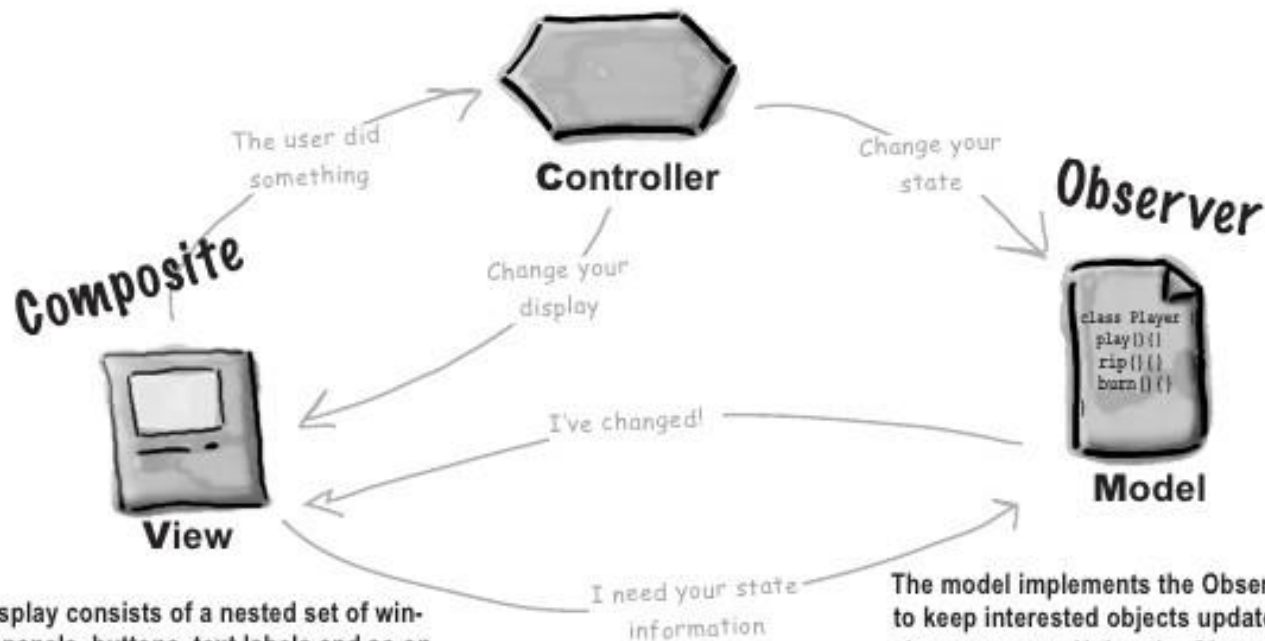
Strategy

Strategy



Strategy

The view and controller implement the classic Strategy Pattern: the view is an object that is configured with a strategy. The controller provides the strategy. The view is concerned only with the visual aspects of the application, and delegates to the controller for any decisions about the interface behavior. Using the Strategy Pattern also keeps the view decoupled from the model because it is the controller that is responsible for interacting with the model to carry out user requests. The view knows nothing about how this gets done.



The display consists of a nested set of windows, panels, buttons, text labels and so on. Each display component is a composite (like a window) or a leaf (like a button). When the controller tells the view to update, it only has to tell the top view component, and Composite takes care of the rest.

The model implements the Observer Pattern to keep interested objects updated when state changes occur. Using the Observer Pattern keeps the model completely independent of the views and controllers. It allows us to use different views with the same model, or even use multiple views at once.

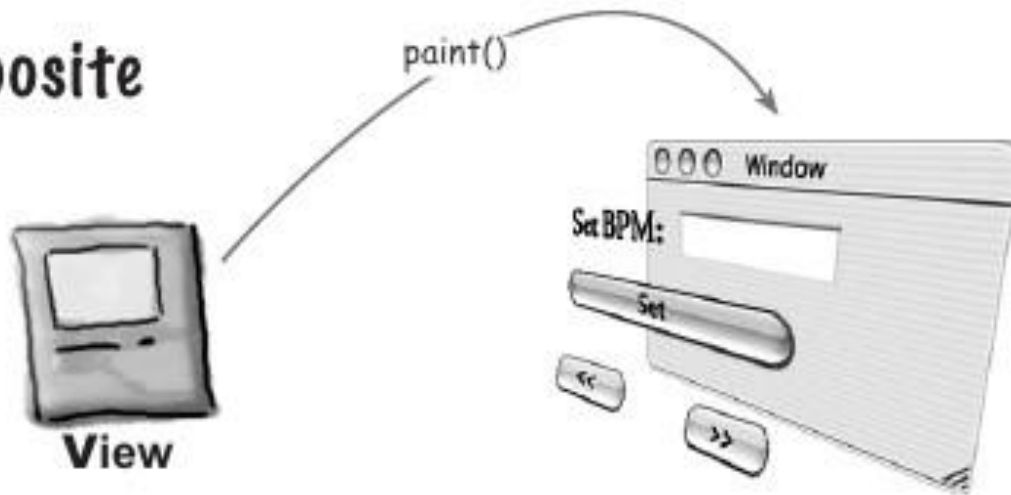
From "Head First Design Patterns"

Composite Pattern Motivation

- Software System:
 - Definition: A software system consists of subsystems which are either other subsystems or collection of classes
 - Composite: Subsystem (A software system consists of subsystems which consists of subsystems , which consists of subsystems, which...)
 - Leaf node: Class
- Models tree structures that represent part-whole hierarchies with arbitrary depth and width.
- The Composite Pattern lets client treat individual objects and compositions of these objects uniformly

Composite

Composite



The view is a composite of GUI components (labels, buttons, text entry, etc.). The top level component contains other components, which contain other components and so on until you get to the leaf nodes.

MVC in Web Applications

- Model: application domain data or business logic. (Often directly related to DB, knowing nothing about html, or web server)
- View: UI. Usually can query model to display, but not supposed to change model. It can be implemented using a template that renders an html page
- Control: handle user actions. Receives user requests and translate them into actions that model should take. It can also select appropriate view to handle the response – it can handle the incoming http requests. (e.g. common actions like authentication, session management)
- Decouple model and view, so that the presentation is always independent of application data
- Decouple view and control so that the request made by the user can be independent of the resulting page.

MVC Web Framework

- ASP .NET Framework (...)
- Spring framework (...) (Java)
- CodeIgniter (...) (php)
- Ruby on Rails (...) (Ruby)
- Django (...) (Python)
- http://en.wikipedia.org/wiki/Comparison_of_web_application_frameworks

MVC Web Applications in PHP

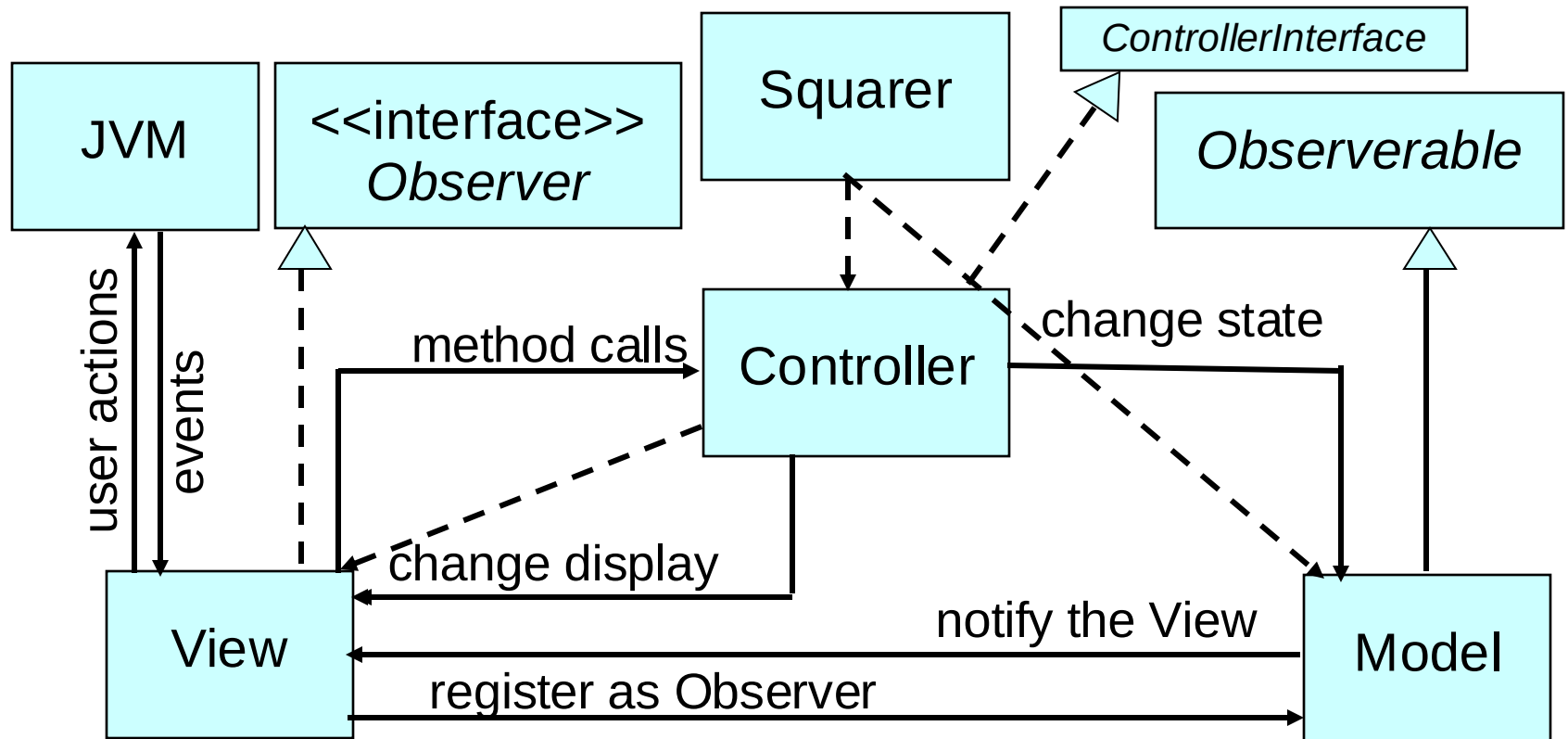
- Model – those functions that wrap calls to your db
- View – the templates / scripts that output HTML
- Controller – the stuff that examines variables like `$_GET` and `$_POST` and works out what to do next

Controller

- Controller should be as minimal as possible.
 - It is only responsible for translating HTTP requests into Model actions and selecting the right View
 - The Model should provide all the behavior it can without handling the HTTP requests or output formatting details

The MVC Pattern: Summary

- The UML diagrams for this design:
 - user actions are passed from the View to JVM, back to View, then to the Controller

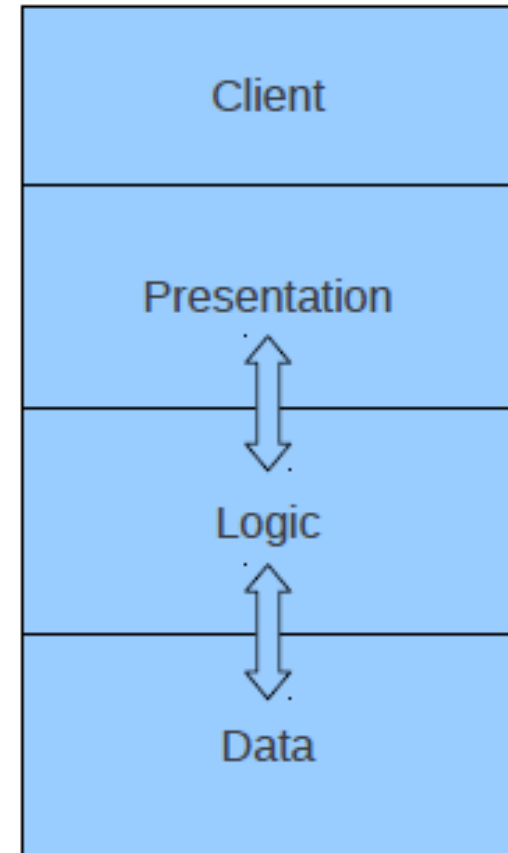


The MVC Pattern: Summary

- Summary
 - the MVC compound pattern has the Strategy, Observer, and Composite patterns with the Adapter as a possibility
 - the Controller is the strategy for the view (different implementations can be used)
 - the Model represents the problem domain; it notifies its observer View but is decoupled from it and from the Controller
 - the View is a composite of nested composites and their components

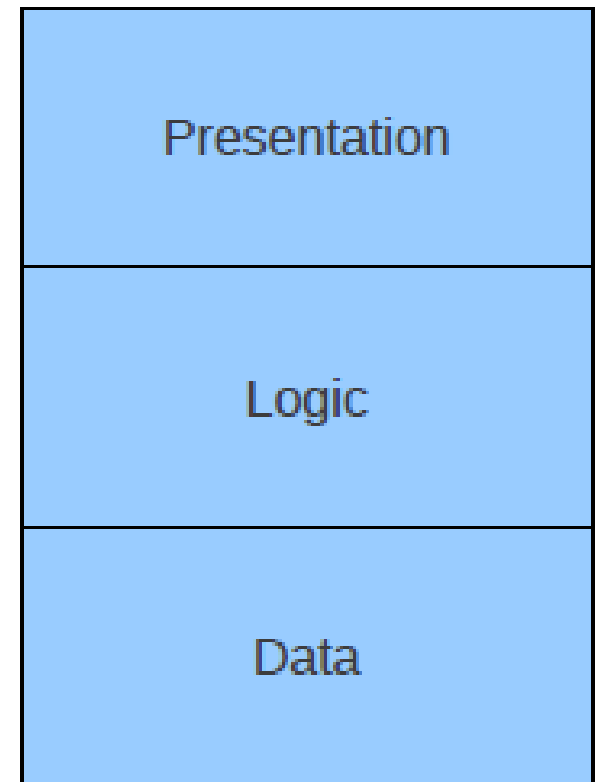
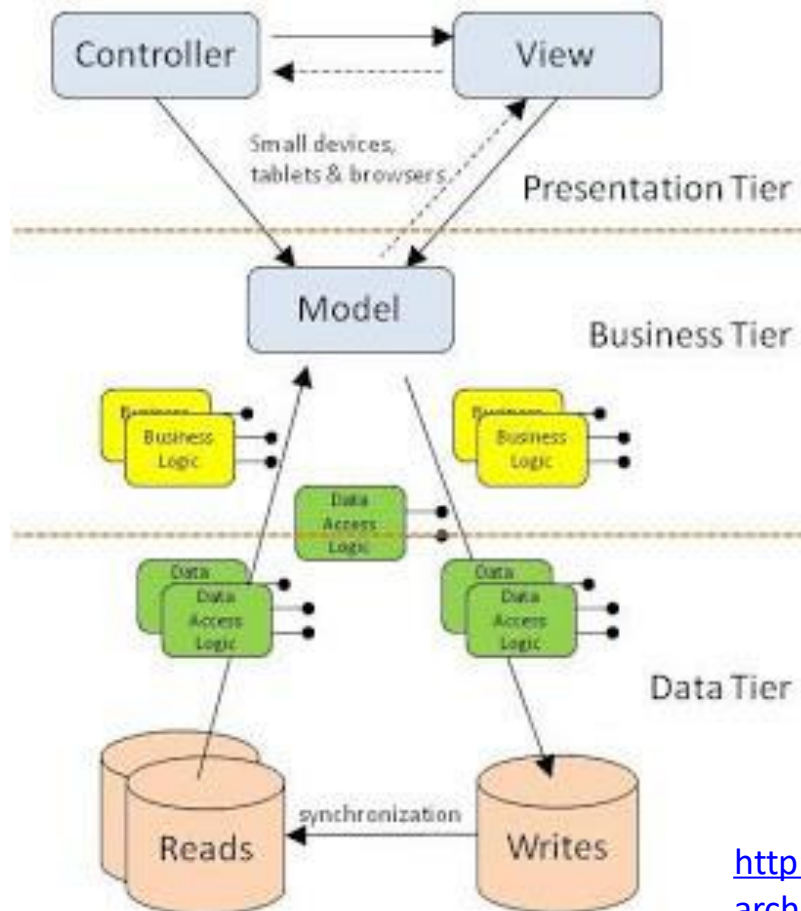
Three/Four -Tier Architecture

- Presentation: UI
- Logic: business logic
- Data: database/file
- Performance, security, reuse, maintainability and understandability.



MVC vs. Three-Tier Architecture

- Three-tier Architecture



<http://criticaltechnology.blogspot.com/2011/10/mvc-in-three-tier-architecture.html>